

SUBCELLULAR LOCALIZATION OF ANIMAL AND HUMAN PROTEINS USING MACHINE LEARNING ALGORITHMS

by

Benjamin Annan

A Project submitted in Partial Fulfilment of the Requirements for the
Degree of

MASTER OF SCIENCE

in the

Department of Mathematics and Statistics

Graduate Project Committee

Dr. G. Andy Chang (Chair)
Department of Mathematics and Statistics

Dr. Xiangjia Jack Min
Department of Biological Sciences

Dr. G. Jay Kerns
Department of Mathematics and Statistics

YOUNGSTOWN STATE UNIVERSITY
May 2020

Abstract

Predicting the subcellular location of animal and human protein has been a major problem faced by researchers in biology due to its tedious laborious work, high cost and even time consuming. This has resulted in subcellular localization prediction becoming one of the challenges being successfully aided by bioinformatics, and machine learning. Proteins are able to function very well when their subcellular locations are accurately predicted. In general, prediction tools take as input information about a protein, such as a protein sequence of amino acids, which produce a predicted location within the cell as end product, such as the nucleus, Endoplasmic reticulum, Golgi apparatus, Cytoplasm, Secreted, Lysosome, Plasma membrane, Vacuole, mitochondrial and Peroxisome. The Objective of this study is to build tools (classifiers) that can accurately predict the outcome of animal and human protein targeting in cells (Subcellular locations) and also to determine which one of the classifiers is the best predictor. source of data for this research was from the The Universal Protein Resource (UniProt), a database of glossary protein sequence. This database was carefully reviewed to consider some of the animal and human proteins used in this study. A benchmark data set comprising of 10 various protein subcellular locations were obtained after employing the method outlined in this study. These are nucleus, Endoplasmic reticulum, Golgi apparatus, Cytoplasm, Secreted, Lysosome, Plasma membrane, Vacuole, mitochondrial and Peroxisome. pseudo-amino acid composition was the technique employed from past research in representing proteins. Adaptive Boosting, Bagging, Cross Validated Bagging, Random Forest, Extreme Gradient Boosting, Deep Neural Network and Automatic Machine Learning were the prediction algorithms employed in this study in predicting the subcellular location of animal and human proteins. study showed that, the Automatic Machine Learning had the best performance in terms of the overall prediction accuracy. This was followed by Random Forest with up sampling and Extreme Gradient Boosting with down sampling being the worse predictor. The automatic machine learning had a very good prediction in determining the plasma subcellular location with a perfect MCC and Sensitivity value. Even though the other algorithms had good and

moderate predictions, their prediction accuracy can still be improved in future study.

Acknowledgement

I have taken efforts in this project. However, it would not have been possible without the kind support and help of many individuals, the Mathematics and Bio informatics Department and the university as a whole. I would like to extend my sincere thanks to all of them. am highly indebted to Dr. Andy Chang (Mathematics Department) for his statistical guidance and Dr. Jack Min (Bioinformatics) for providing me with data and also addressing issues concerning the dataset.

Also I will like to acknowledge Dr Kerns for his indirect guidance during the time of this project. would like to express my gratitude towards my mother and the rest of my family for their ever ending prayer during my graduate studies. would also like to express my special gratitude and thanks to my predecessors for giving me such attention and time. My thanks and appreciations also go to my colleagues in developing the project and people who have willingly helped me out with their abilities.

Contents

1	Introduction	9
2	Methodology	14
2.1	Method of Protein representation	14
2.2	Dataset	16
2.3	Re sampling	19
2.3.1	Under-sampling/ Down-sampling	19
2.3.2	Over-sampling / Up-sampling	19
2.4	Classification Methods	19
2.4.1	Bootstrap Aggregating (Bagging)	20
2.4.2	Random Forest	21
2.4.3	Adaboost	22
2.4.4	Gradient Boosting	24
2.4.5	Artificial Neural Networks (Deep Learning)	25
2.4.6	Deep Neural Network	29
2.4.7	Automatic Machine Learning (AutoML)	30
3	Results	31
3.1	Key Terms	31
3.2	Adaptive Boosting Model	33
3.3	Adaptive Boosting Model (Down Sampling)	35
3.4	Adaptive Boosting Model (Up Sampling)	37
3.5	Bagging	39
3.6	Bagging (Down Sampling)	41
3.7	Bagging (Up Sampling)	43
3.8	Cross Validated Bagging	45
3.9	Cross Validated Bagging (Down Sampling)	47

3.10	Cross Validated Bagging (Up Sampling)	49
3.11	Random Forest	51
3.12	Random Forest (Down Sampling)	53
3.13	Random Forest (Up Sampling)	55
3.14	XGBoost	57
3.15	XGBoost (Down Sampling)	59
3.16	XGBoost (Up Sampling)	61
3.17	Deep Neural Network	63
3.18	Automatic Machine Learning (AutoML)	64
4	Discussion	68
5	Conclusion	71
	References	117

List of Tables

1	Overall Statistics	33
2	Statistics by Class	33
3	Overall Statistics	35
4	Statistics by Class	35
5	Overall Statistics	37
6	Statistics by Class	37
7	Overall Statistics	39
8	Statistics by Class	39
9	Overall Statistics	41
10	Statistics by Class	41
11	Overall Statistics	43

12	Statistics by Class	43
13	Overall Statistics	45
14	Statistics by Class	45
15	Overall Statistics	47
16	Statistics by Class	47
17	Overall Statistics	49
18	Statistics by Class	49
19	Overall Statistics	51
20	Statistics by Class	51
21	Overall Statistics	53
22	Statistics by Class	53
23	Overall Statistics	55
24	Statistics by Class	55
25	Overall Statistics	57
26	Statistics by Class	57
27	Overall Statistics	59
28	Statistics by Class	59
29	Overall Statistics	61
30	Statistics by Class	61
31	Overall Statistics	63
32	Statistics by Class	63
33	Overall Statistics	64
34	Statistics by Class	64
35	Comparison of best 3 prediction accuracies of animal protein subcellular lo- cation by different algorithms	66
36	Best Models For Each Subcellular Location	67

List of Figures

1	Animal Cell	12
2	Structure of a Neuron	26
3	A single multiple input neuron	27
4	A Deep Neural Network	30

1 Introduction

Cells are the makeup of every living organism. A cell is a small membrane enclosed structure, filled with an aqueous solution where organelles and other subcellular structures are found. Cells are of different sizes and shapes. Their different genetic appearances can be traced to its specific function. Every single cell comprises of a fluid called the cytoplasm, which is contained in a membrane. The cytoplasm also contains several biomolecules like proteins, nucleic acids and lipids. Furthermore, cellular structures called cell organelles are suspended in the cytoplasm.

Proteins on the other hand are by far the most structurally complex and functionally sophisticated molecules known. Proteins contain 20 types of amino acids in them which come with unique chemical characteristics. A protein molecule is made from a long chain of these amino acids, each linked to its neighbor through a covalent peptide bond. Proteins are therefore also known as polypeptides. Every type of protein has a unique sequence of amino acids, exactly the same from one molecule to the next (Alberts et al., 2002). Knowing the exact location of a particular novel protein within a cell can lead to a better understanding of its function, or even lead to suggested experiments to test function (Meissner et al., 2011).

Cytoplasm is itself a significant contributor to cell structure. In eukaryotes, it fills the space between the nucleus and the cell membrane. The fluid portion, excluding the organelles, is called cytosol and consists mostly of water. The makeup of cytoplasm shows that water is necessary for maintaining cell structure. This is only one of many reasons that water is an essential component for life. However, Many chemical reactions occur in the cytoplasm, where water acts as an important solvent.

Nucleus is the storehouse for most of the genetic information, or DNA (deoxyribonucleic acid) and DNA contains genes that are instructions for making proteins. There are two major demands on the nucleus: firstly, DNA must be carefully protected, and it must be available for use at the proper times. Molecules that would damage DNA need to be kept out

of the nucleus. But many proteins are involved in turning genes on and off, and they need to access the DNA at certain times. The special structure of the nucleus helps it meet both demands. The nucleus is composed of the cell's DNA enclosed in a double membrane called the nuclear envelope. Each membrane in the nuclear envelope is similar to the membrane surrounding the entire cell. The nucleus also contains the nucleolus. The nucleolus is a dense region where tiny organelles essential for making proteins are assembled. These organelles, called ribosomes, are a combination of proteins and RNA molecules.

A large part of the cytoplasm of most eukaryotic cells is filled by the endoplasmic reticulum, the endoplasmic reticulum, or the ER is an interconnected network of thin, folded membranes. The composition is very similar to that of the cell membrane and nuclear membranes. The ER membranes form a maze of enclosed spaces. The interior of this maze is called the lumen. Numerous processes, including the production of proteins and lipids, occur both on the surface of the ER and inside the lumen.

From the ER, proteins generally move to the Golgi apparatus. The Golgi apparatus consists of closely layered stacks of membrane-enclosed spaces that process, sort, and deliver proteins. Its membranes contain enzymes that make additional changes to proteins. The Golgi apparatus also packages proteins. Some of the packaged proteins are stored within the Golgi apparatus for later use. Some are transported to other organelles within the cell and others are still carried to the membrane and secreted outside the cell.

Mitochondria supply energy to the cell. Mitochondria (singular, mitochondrion) are bean shaped and have two membranes. The inner membrane has many folds that greatly increase its surface area. Within these inner folds and compartments, a series of chemical reactions converts molecules from the food we eat into usable energy. They are also organelles that are responsible for energy transformation. They work and break down carbohydrates which releases adenosine triphosphate (ATP) molecules. ATP is the carrier of energy in the cell. Cells use ATP to synthesize and transport molecules and also perform specific functions.

A vacuole is a fluid-filled sac used for the storage of materials needed by a cell. These materials may include water, food molecules, inorganic ions, and enzymes. Most animal cells contain many small vacuoles. Lysosomes are membrane-bound organelles that contain enzymes. They defend a cell from invading bacteria and viruses. They also break down damaged or worn-out cell parts. Lysosomes tend to be numerous in animal cells. The Plasma membrane is a membrane that encloses the cell and separates the interior of the cell from its external environment. Peroxisomes are small, organelles that contain enzymes involved in a variety of metabolic reactions, including several aspects of energy metabolism.

A cell contains 109 proteins located in different parts of the cell on average. Figure 1 illustrates the different compartments and organelles in the cell (Chou & Shen, 2007).

Subcellular localization is where the cell is being located. In cell biology and proteomics, one of the most significant aim is to predict the subcellular location of different proteins and their functions in the cell. Protein subcellular localization prediction (or just protein localization prediction) involves the prediction of where a protein resides in a cell.

In general, prediction tools helps to bring out information about a protein, such as a protein sequence of amino acids, and also produce a predicted location within the cell as output, such as the nucleus, Endoplasmic reticulum, Golgi apparatus, extracellular space, or other organelles. The aim is to build tools that can accurately predict the outcome of protein targeting in cells.

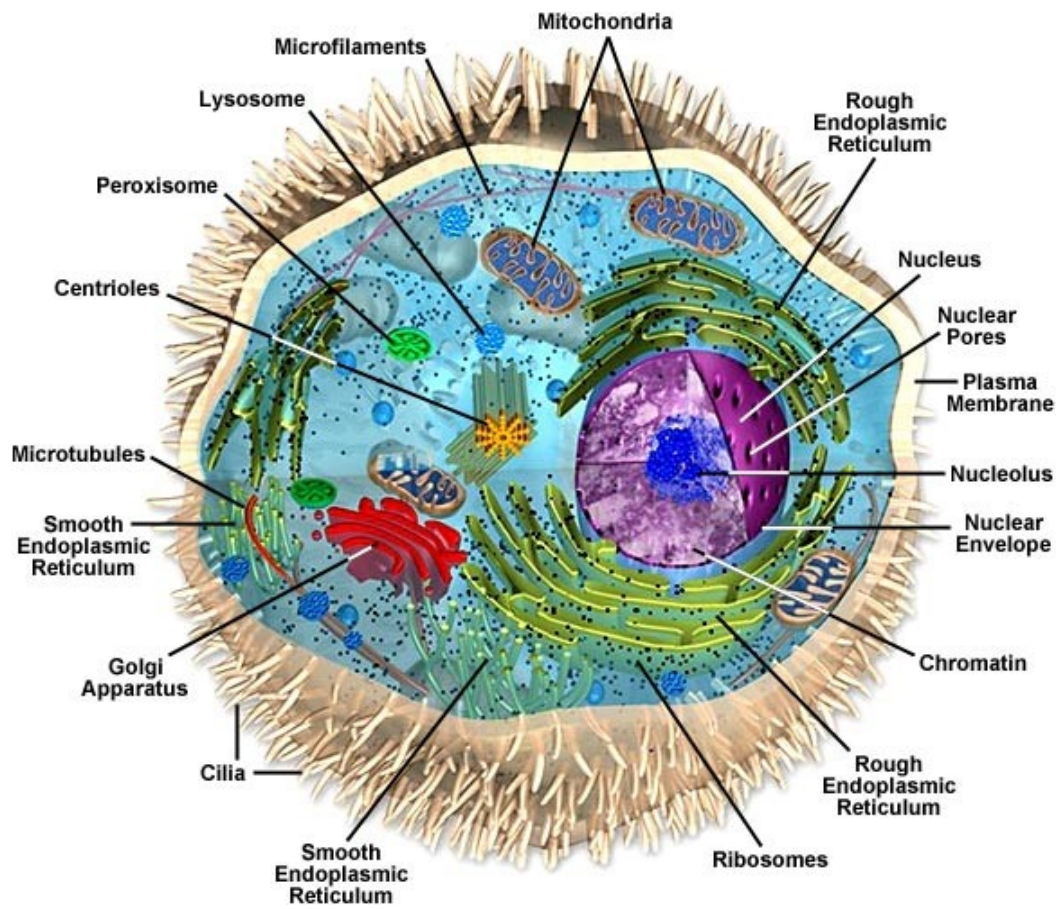


Figure 1: Animal Cell
source:<https://micro.magnet.fsu.edu/cells/animals/images>

Generally, there are two approaches for the protein subcellular localization problem: experimental methods and computational methods. Several experimental approaches such as green fluorescent protein (Webb, Decatur, Teleman, & Losick, 1995), microscopic detection (Glory & Murphy, 2007) and subcellular proteomics (Chou & Shen, 2010) have been already introduced to identify subcellular locations of a protein. Unfortunately, experimental methods are time consuming and costly. That is why a large information gap exists between protein sequences and their location. This gap grows by the day. Consequently, various computational methods have been developed to fill this gap (Emanuelsson, 2000).

Computational methods have their advantages and disadvantages. These methods outperform experimental methods, both in terms of time and cost, but they may not be as accurate as experimental methods. Moreover, most of these computational methods focus on the single site SCL of a protein whereas the experimental researches show that many proteins are located in several subcellular locations (Chou & Shen, 2007). On the other hand, most of these methods are developed for particular proteins or species (Chou & Shen, 2006). Hence, it seems that a more comprehensive method is desired to predict multiple locations for various proteins while remaining applicable to different species.

Therefore, the main objective of this study is to use several different classifiers to predict the subcellular location of animal and human proteins and to determine which of these classifiers performs the best in predicting protein subcellular location.

2 Methodology

2.1 Method of Protein representation

Protein representation is an essential part in predicting its subcellular locations. Several methods have been adopted in performing this task. In this study, the sequential and non-sequential methods were employed to represent the protein samples. This and many methods have been highlighted the literature review. Pseudo amino acid composition (PseAA composition) method was used in this study to represent the protein data which was affirm from .This method of pseudo amino acid composition is non sequential in representing protein data. The sequential method fall short in its performance when there is a small coordination to protein concern in their known location .This makes the non sequential method more beneficial over the sequential method and Amino acid composition method (AA composition). The Amino acid composition method also disconnect its protein order of sequence (Chou & Shen, 2007).

Chou and Shen describes the way protein representation are made by the Pseudo Amino acid composition (PseAA Composition).

PseAA Composition (Chou & Shen, 2007):

Consider a query protein sequence P that comprised of L amino acid residues. In the AA composition discrete model this protein would be represented as

$$P = R_1, R_2, R_3, R_4,R_L \quad (1)$$

R_1 from the equation above represents the first residue from protein P, R_2 represents the

second residue and this pattern continues to R_L being the L th residue. The above protein representation can be expressed according to the Amino Acid (AA)

$$P = [f_1, f_2, \dots, f_{20}]^T \quad (2)$$

where $f_u (u = 1, \dots, 20)$ are the normalized occurrences of the 20 native amino acids.

It is necessary to note the demerit of AA composition which makes the order of the amino acid representation in the original protein is lost. The substitute to this is the pseudo amino acid (Pse AA) composition which retains order of amino acids in the original protein.

In the Pse AA composition model the query protein sequence is represented by:

$$P = [p_1, p_2, \dots, p_{20}, p_{20+1}, \dots, p_{20+\lambda}]^T, (\lambda < L) \quad (3)$$

In this equation, the $20 + \lambda$ components are given by:

$$P_u = \begin{cases} \frac{f_u}{\sum_{i=1}^{20} f_i + w \sum_{k=1}^{\Lambda} T_k}, & (1 \leq \mu \leq 20) \\ \frac{wT_u - 20}{\sum_{i=1}^{20} f_i + w \sum_{k=1}^{\Lambda} T_k}, & (20 + 1 \leq \mu \leq 20 + \Lambda) \end{cases} \quad (4)$$

W in the above equation denotes the weight factor which is set to 0.05 according to Chou and Shen (2007). The T_k on the other hand is the k^{th} tier correlation factor which represents the sequence order correlation between all of the k^{th} most adjacent residues. The parameter Λ is the number of components in the protein's PseAA composition representation. As the parameter gets bigger, the number of components representation in the protein gets greater and the value of Λ cannot exceed the number of amino acids in a protein (Chou & Shen, 2007).

This study takes into account the occurrence frequencies amino acid of 20 and the parameter $\Lambda = 15$ was selected according to studies from (Neizer-Ashun, Yu, Meinken, Min, & Chang, 2015) and (Munyon, 2015).

2.2 Dataset

The data for this study was obtained from The Universal Protein Resource (UniProt) which is a database of protein sequence and annotation (Consortium, T. U et al., 2015). The dataset in this paper are animal and human proteins.

The following criteria were implemented to retrieve the data in order to obtain an efficient benchmark dataset as prescribe in the literature for benchmark dataset.

- All proteins present in this dataset are examined and annotated for a subcellular location.
- Protein sequence having length equal or more than 100 amino acids were retain, which means protein sequences with length less 100 amino acids were omitted.
- Protein sequences with unclear and insecure labels such as 'probable' or 'by similarity' have been ruled out
- All proteins in the dataset are examined only for one subcellular location(protein sequences that have multiple subcellular locations have been excluded from the dataset).
- Sequences labeled 'fragments' in the database were as well eliminated. That is, all protein sequences not starting with 'M'(indicating fragments) were omitted

After applying the above criteria to the original dataset, 80/80 BLASTClust (Developmental Biology, M.-P. I.) is used on the dataset. The purpose of using 80/80 BLASTClust is to cluster similar proteins together and to choose one proteins at random in each cluster for the final dataset. The rule for similarity of proteins is that if two or more proteins are at least 80% similar over at least 80% of their length, then they belong to the same cluster.

After applying 80/80 BLASTClust to the dataset, the working benchmark dataset includes 52290 protein sequences. The subcellular locations of these proteins are the nucleus (12403 proteins), the cytoplasm (9759 proteins), the secreted (17581 proteins), the mitochondria (7629 proteins), Plasma membrane (266 proteins), the endoplasmic reticulum (2303 proteins), the Golgi apparatus (1364 proteins), the peroxisome (351 proteins), lysosome (605 proteins) and vacuole (29 proteins). Therefore, there are 10 different subcellular locations for proteins in this dataset.

The benchmark dataset is partitioned into two parts: a training (learning) dataset for training the classifiers and testing dataset for evaluating the classifiers. The training dataset is made of 70% of the benchmark dataset and includes a total of 36599 proteins, 186 in the plasma membrane, 6831 proteins in the cytoplasm, 423 proteins in the lysosome, 1612 proteins in the endoplasmic reticulum, 954 in the Golgi apparatus, 5340 proteins in the mitochondria, 8682 in the nucleus, 245 in the peroxisome, 12306 in the secreted, 20 proteins in the vacuole. The test dataset is made of 30% of the benchmark dataset and includes a total of 15691 proteins, 80 in the plasma membrane, 2928 proteins in the cytoplasm, 182 proteins in the lysosome, 691 proteins in the endoplasmic reticulum, 410 in the Golgi apparatus, 2289 proteins in the mitochondria, 3721 in the nucleus, 106 in the peroxisome, 5275 in the secreted, 9 proteins in the vacuole.

Prior to predicting the subcellular location of proteins in this study, various methods of classification are discussed. In the following sections, methods of Bootstrap Aggregating, Random Forest, Adaptive Boosting, and Gradient Boosting Machines examined. The R

software environment and other R packages were used to apply the classification methods to predict the protein subcellular location (Josefine Sprenger and J Lynn Fink, R. D. T, 2006).

2.3 Re sampling

This is a method that consist of drawing repeated samples from the original data samples. The method of Resampling is a non-parametric method of statistical interference. The literature review discussed numerous ways in dealing with disproportion in dataset. Eliminating samples from the majority class or adding samples to the minority class are the main objective of this method.

2.3.1 Under-sampling/ Down-sampling

During this process, random samples are remove from the majority class, with or without replacement. This is one of the earliest techniques used to alleviate imbalance in the dataset, however, it may increase the variance of the classifier and may potentially discard useful or important samples (Chawla et al., 2018).

2.3.2 Over-sampling / Up-sampling

Random Oversampling involves supplementing the training data with multiple copies of some of the minority classes. Oversampling can be done more than once (2x, 3x, 5x, 10x, etc.) This is one of the earliest proposed methods that is also proven to be robust. [27] Instead of duplicating every sample in the minority class, some of them may be randomly chosen with replacement.

2.4 Classification Methods

Issues concerning classifications can be tackle with different approaches that includes decision trees, rule-based methods, probabilistic methods, Support Vector Machine (SVM) methods, instance-based methods, neural networks or using ensemble learning etc. In other instances,

the Ensembling method can also be adopted to build decision trees by allowing each trees vote for a class then the most popular will be picked for the final class in the classification problem. The Bootstrap Aggregation method is an example ensembling . This method was first introduced by Leo Breiman (Breiman et al., 1996a) and (Breiman et al., 1996b).Selecting split from the best random of each best node came about by (Yali Amit, D. G., 1997)] which start with a large number of geometric features and use a random selection of these features to find the best split at each node.Also Breiman (Breiman L. et al., 1999) offers another method in building new training sets by randomizing output in the original data set.Random Forest, Bagged Trees, AdaBoost, and Gradient Boosting (XGBOOST) are some of the classification approach employed in this study.

Furthermore, we will outlined and explain some of these methods in relation to classification and point out some of their merit and demerits. Also each method of classification is applied in predicting the subcellular location of proteins in this study. Then in conclusion, the results obtained from each classification method will be examined.

2.4.1 Bootstrap Aggregating (Bagging)

Leo Breiman illustrates Bagging as a procedure of classification where different type of predictor are generated and accomplish an accumulator predictor In âBagging Predictors,â. Numerous forms of a predictor are created by making bootstrap replicates of the training set. These replicates are used as new training set. The accumulated predictor equates over the different forms when it is predicting a numerical value and check for a famous (majority) vote to predict a class (Breiman et al., 1996a).Breimn indicated that high accuracy is achieve in solving classification when Bagging is applied on real data In âout-of-bag estimation,â Breiman again showed how advancement in classification accuracy is accomplished by using the bagging method. He indicated that, During constructing bootstrap, replicates of the training set, almost 37 percent of examples in the training set do not show up in a bootstrap training set. This means that since examples are not used in the test, accuracy estimates

may be an alternative way. Breiman refers to the unused examples as "out-of-bag estimates". He indicated that the alternative way to utilize these estimates to generalized estimate the generalized error, strength, and correlation of classifiers. In addition He also combined cross-validating bagged predictors which requires a lot of computing power. "out-of-bag estimates" computing ability are more efficient (Breiman et al., 1996b). It is significant to note that the brain behind "Bagging" add to the development of the random forest classification method.

2.4.2 Random Forest

Random forests or random decision forests are an ensemble learning method for classification, regression and other tasks that operate by constructing a multitude of decision trees at training time and outputting the class that is the mode of the classes (classification) or mean prediction (regression) of the individual trees (Ho, Tin Kam et al., 1995).

This approach was introduced by Leo Breiman (Breiman L. et al., 2001) and Adele Cutler (Liaw, A. et al., 2012) and the main concept was to establish numerous tree models that will enhance predicting classes and joining all these models to create an exclusive model. Each and every tree produces a classification and that class is voted for but the tree. The forest then selects the classification having majority of votes among the various trees. Bootstrapping of data produces the ultimate split of data at each level of building individual trees. The accuracy model produce by esembling in random forest reduces the instability in a single decision tree and make the approach even more powerful to data dynamics which in the longrun is not affected by outliers and noise . The post "classification Random Forest error rate relies on two major factors. These factors are the correlation two different trees and how well they withstand each individual tree in it. Here there is a positive relationship between forest error with a tree with low rate that is a strong classifier. Also a negative relationship exist between individual trees and forest error rate. Breiman defines a random forest is a classifier consisting of a collection of tree-structured classifiers

$\{ h(x, \Theta_k), k = 1, \dots \}$ where Θ_k the are independent identically distributed random vec-

tors and each tree casts a unit vote for the most popular class at input x (Breiman L. et al., 2001). Breiman also explains the steps of Random Forest by noting that for the k^{th} tree, a random vector Θ_k is generated independent of past vectors $\Theta_1, \Theta_2, \Theta_3, \dots, \Theta_{k-1}$ but with the same distribution. A tree is grown using the training set and the random vector Θ_k which results in a classifier $h(x, \Theta_k)$ (Breiman L. et al., 2001). By following this procedure, one can construct a large number of trees. These trees will each vote for a class and the most popular class is chosen as the final class (Breiman L. et al., 2001).

According to Breiman, the Random Forest has an accuracy competitive with and sometimes better than Adaboost, its accuracy is not affected by outliers and noise in the data, the algorithms run faster than bagging or boosting, and they generate out-of-bag estimates which include the internal estimate of error, strength, and correlation (Breiman L. et al., 2001).

2.4.3 Adaboost

Boosting refers to a family of algorithms which converts weak learner to strong learners. Boosting is an ensemble method for improving the model predictions of any given learning algorithm. The idea of boosting is to train weak learners sequentially, each trying to correct its predecessor as various authors have shown both theoretically and empirically (Bald. et al., 2007) and (Bauer, E. and J Kohavi R, 1999). Yoav Freund and Robert Schapire popularized adaptive boosting which is a machine learning meta-algorithm. Adaboost is a type of boosting that combines multiple weak learners into a single strong learner. The weak learners in AdaBoost are decision trees with a single split, called decision stumps. When AdaBoost creates its first decision stump, all observations are weighted equally. To correct the previous error, the observations that were incorrectly classified now carry more weight than the observations that were correctly classified. This approach was the first practical boosting algorithm proposed by Freund and Schapire (1996). It focuses on classification problems and aims to convert a set of weak classifiers into a strong one.

The algorithm of Adaboost by Freund and Schapire(1996) [39] was implemented by the following procedure.

Adaboost Algorithm

1. Initialize the observation weight $W_i = \frac{1}{n}, i = 1, \dots, n$
2. For $m = 1$ to M :
 - Fit a classifier $T^m(X)$ to the training data using weights W_i
 - Compute $err^m = \sum_{i=1}^N W_i I(c_i \neq T^m(X)) / \sum_{i=1}^N W_i$
 - Compute $\alpha^m = \log \frac{1-err^m}{err^m}$
 - Set $w_i \leftarrow w_i \exp(\alpha^m \cdot I(c_i \neq T^m))$, $i = 1, \dots, n$
 - Re normalize w_i
3. Output $c(x) = \operatorname{argmax} \sum_{m=1}^M \alpha^m I(T^m(x) = k)$

2.4.4 Gradient Boosting

Gradient boosting is a machine learning technique for regression and classification problems, which produces a prediction model in the form of an ensemble of weak prediction models, typically decision trees. It builds the model in a stage-wise fashion like adaboost and other algorithms. The idea of gradient boosting originated in the observation by Leo Breiman that boosting can be interpreted as an optimization algorithm on a suitable cost function (Breiman L. et al., 1997).

Breiman duplicated Adaboost and other similar algorithms into a statistical framework that was named ARcing meaning Adaptive Reweighting and Combining. In this approach each step comprises of a weighted minimization and a computation of the classifiers and weighted input.

This framework was further refined by Friedman and called Gradient Boosting Machines now gradient tree boosting.

The major aim of this statistical framework is to lower the loss of the model adding weak learners with the help of a known approach. This refines boosting as a numerical optimization. Here, the parameters to reduce the cost function is the main notion.

During batch gradient descent, the entire training set is used to identify the gradient at every level of iteration. During the process, error gradient and convergence are fixed and also takes a while in a huge data set.

How Gradient Boosting Works

1. A loss function to be optimized.
2. A weak learner to make predictions - Decision Trees are used as weak learners
3. An additive model to add weak learners to minimize the loss function.

General Steps for gradient boosting

1. A model is built on a subset of the training data.
2. Using this model, predictions are made on the whole training data set.
3. Errors are calculated by comparing the predictions and actual values.
4. A new model is created using the errors calculated as target variable. Our objective is to find the best split to minimize the error.
5. The predictions made by this new model are combined with the predictions of the previous.
6. New errors are calculated using this predicted value and actual value.

This process is repeated until the maximum limit of the number of estimators is reached. XGBoost is an implementation of gradient boosted decision trees designed for speed and performance.

2.4.5 Artificial Neural Networks (Deep Learning)

This is a connectionist systems inspired by the biological neural networks that constitute animal brains. Such systems learn (progressively improve their ability) to do tasks by considering examples, generally without task-specific programming. An ANN is based on a collection of connected units called artificial neurons. Each connection (synapse) between neurons can transmit a signal to another neuron. The receiving (postsynaptic) neuron can process the signal(s) and then signal downstream neurons connected to it. Neurons may have state, generally represented by real numbers, typically between 0 and 1. Neurons and synapses may also have a weight that varies as learning proceeds, which can increase or

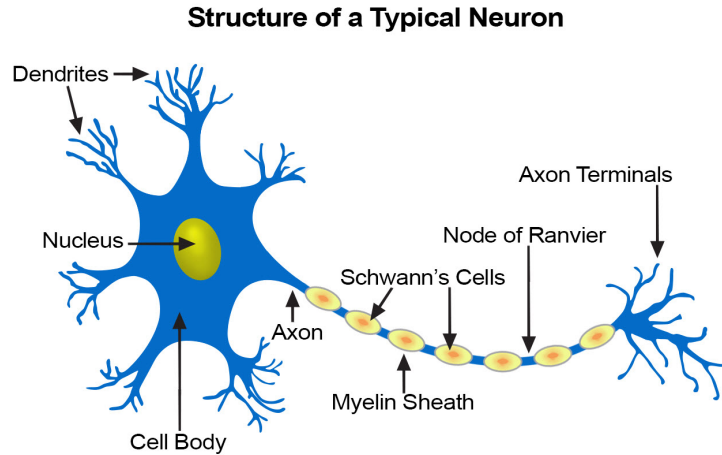


Figure 2: Structure of a Neuron
source:<https://images.app.goo.gl/iVd6UQ4MGTFVWjED7> [45]

decrease the strength of the signal that it sends downstream.

A weighted directed graph can also be used to demonstrate an artificial neural network. In this case, the artificial neurons are the nodes of the graph and the connections between neuronsâ inputs and outputs are the directed edges (these directed edges also include weights) [43]. All neurons in the network are similar in structure and contains some addition of a function unit. The inputs are then fed into the neural network, which assigns weight to each input. Next, the weighted inputs are all added up. The result of this step is used as an input for the transfer or activation function in the neuron. A common neural network is comprised of three layers which comprises of an input layer, a hidden layer, and an output layer. The Input layer comprises of initial data for the neural network. The Hidden layers on the other hand is the intermediate layer between input and output layer and place where all the computation is done. Finally the Output layer produces the result for given inputs. Figure 3 below depicts the structure of the three layers.

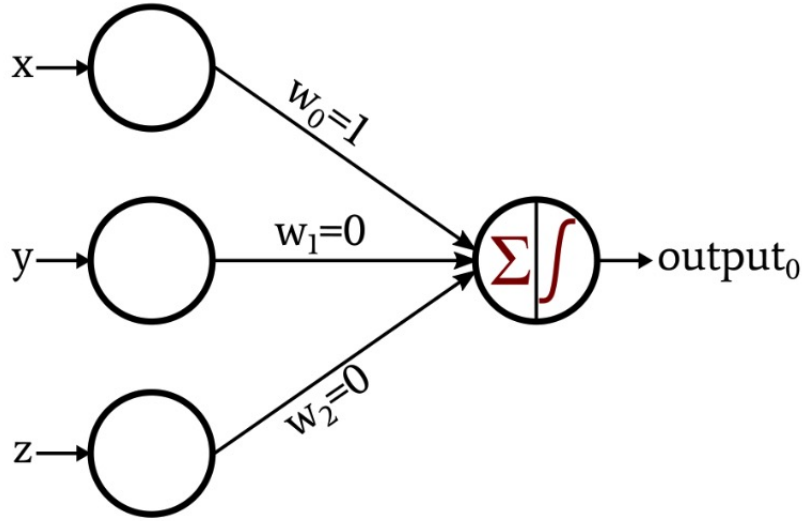


Figure 3: A single multiple input neuron
source:<https://images.app.goo.gl/iVd6UQ4MGTFVWjED7> [45]

The output from the activation function is the output of the neuron. There are different types of activation functions that can be used in a neural network .An activation function is a mathematical function which converts the input to an output. There are many activation functions some of which are outlined below.

1. The linear activation function represented by $y = f(x) = x$.The output is same as the input and the function is defined in the range $(-\infty, +\infty)$
2. Unit step activation function represented by

$$f(x) = \begin{cases} 0, & \text{if } x < 0 \\ 1, & \text{if } x \geq 0 \end{cases} \quad (5)$$

.They are useful for classifying an output into one of two groups

3. The sigmoid function usually represented by

$$f(x) = \frac{1}{1 + e^{-x}}$$

.It produces an output that ranges between zero and one.

4. The hyperbolic tangent activation function represented by $f(x) = \tanh(x)$, this is a rescaled sigmoid function that produces an output between -1 and +1, and has a steeper slope than the sigmoid function.

5. The Rectified Linear Unit (ReLU) is a very popular function represented by

$$P_u = \begin{cases} 0, & \text{if } x < 0 \\ x, & \text{if } x \geq 0 \end{cases} \quad (6)$$

. if $x \leq 0$.

A perceptron is a single neuron that classifies a set of inputs into one of two categories (usually 1 or -1).

When layers of perceptron are combined together, they form a multilayer architecture, and this gives the required complexity of the neural network processing. Multi-Layer Perceptrons (MLPs) Deep Neural Network (DNN) are created when neural networks are constructed using a two or more hidden layers.

The processing from input layer to hidden layer(s) and then to the output layer is called forward propagation. Once the output is arrived at, at the last layer (the output layer), the error is computed (the predicted output minus the original output). This error is required to correct the weights and biases used in forward propagation. Here is where the derivative function is used. The amount of weight that has to be changed is determined by gradient descent, and the amount by which the weights are adjusted at each iteration (epoch) is known as the learning rate. The process of using the errors to update the weights is known

as back propagation, this continues until some specified criteria has been reached (weights reach a steady value, maximum number of iterations reached or an error function reaching some threshold) .

Neural Networks are used in various applications, like Image recognition, natural language processing, optical character recognition and more.

2.4.6 Deep Neural Network

Deep learning is a class of machine learning algorithms that uses multiple layers to progressively extract higher level features from the raw input. [41]. This is part of a broader family of machine learning methods based on artificial neural networks with representation learning. Learning can be supervised, semi-supervised or unsupervised.[42]. Deep Learning is also based on a multi-layer feedforward artificial neural network that is trained with stochastic gradient descent using back-propagation. The network can contain a large number of hidden layers consisting of neurons with tanh, rectifier, and maxout activation functions. Advanced features such as adaptive learning rate, rate annealing, momentum training, dropout, L1 or L2 regularization, checkpointing, and grid search enable high predictive accuracy. Each compute node trains a copy of the global model parameters on its local data with multi-threading (asynchronously) and contributes periodically to the global model via model averaging across the network.

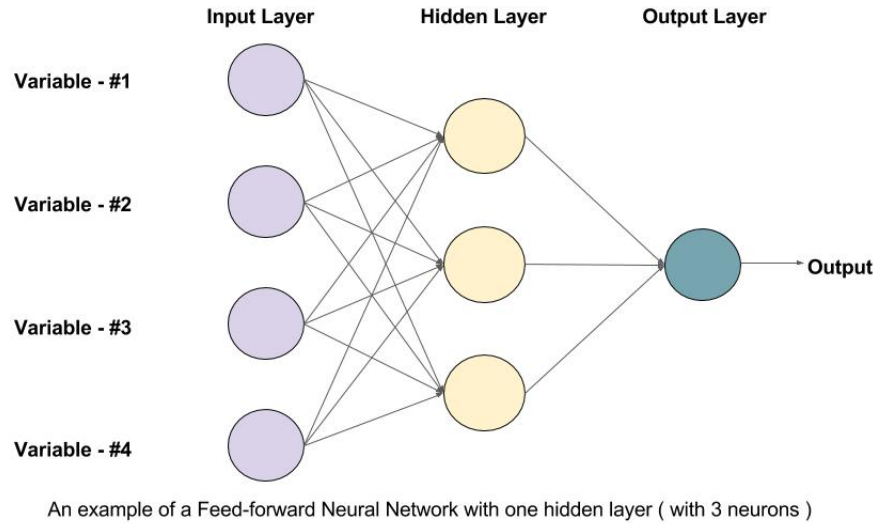


Figure 4: A Deep Neural Network
source:<https://images.app.goo.gl/Y6WcwVF3nMfXJ1k57> [47]

2.4.7 Automatic Machine Learning (AutoML)

This is the process of automating the process of applying machine learning to real-world problems. AutoML covers the complete pipeline from the raw dataset to the deployable machine learning model. AutoML was proposed as an artificial intelligence-based solution to the ever-growing challenge of applying machine learning [44]. AutoML is a function in H2O (a package in R) that automates the process of building a large number of models, The main purpose of this process is finding the "best" model.

This model trains and cross-validates a default Random Forest, an Extremely-Randomized Forest, a random grid of Gradient Boosting Machines (GBMs), a random grid of Deep Neural Nets, a fixed grid of GLMs, and then trains two Stacked Ensemble models at the end. One ensemble contains all the models (optimized for model performance), and the second ensemble contains just the best performing model from each algorithm class/family (optimized for production use).

3 Results

3.1 Key Terms

This section comprises of several essential key words that will be explained before analyzing the results of the study. These key words will be later integrated into discussing the result of the study. One of the key element in measuring performance of a classification method is its Accuracy.

It is one metric for evaluating classification models and can also mean the fraction of predictions our model got right. as Mathewâs correlation coefficient (MCC), sensitivity, specificity ,balanced accuracy etc are several other elements for assessing performance of a predictor or classifier. Below are the various outline of these measurement integrated in classification models.

- TP (True Positives) = the number of true positives (proteins that are predicted to be in a subcellular location and are actually in that location).
- FN (False Negatives) = the number of false negatives (proteins that are not predicted to a subcellular location and are actually in that location).
- FP (False Positives) = the number of false positives (proteins that are predicted to be in a subcellular location but they are not actually in that location).
- TN (True Negatives) = the number of true negatives (proteins that are not predicted to be in a subcellular location and they are not actually in that location).
- The kappa statistic = Is the probability of agreement, which is $(\text{observed agreement} - \text{expected agreement}) / (1 - \text{expected agreement})$. When two measurements agree only at the probability level, the value of kappa is zero. When the two measurements agree perfectly, the value of kappa is 1.

Below are the respective formulas for Mathewâs correlation coefficient, sensitivity, specificity, balanced accuracy (Min, X. J et al., 2010). Both âbalanced accuracyâ and the âMathewâs correlation coefficientâ are used in assessing the performance of classification in machine learning. âBalanced accuracyâ is the average of proportion of sensitivity and specificity. Mathewâs correlation coefficient on the other hand takes into account the true and false positives and negatives and can be used with classes that have different sizes. The Mathewâs correlation coefficient ranges between -1 and 1 with the Mathewâs correlation coefficient value of 1 indicating perfect prediction (Matthews, B et al., 1975).

$$\text{Sensitivity \%} = [TP / (TP + FN)] * 100$$

$$\text{Specificity \%} = [TN / (TN + FP)] * 100$$

$$\text{Balanced Accuracy \%} = [(Sensitivity + Specificity) / 2] * 100$$

$$\text{Mathewâs Correlation Coefficient \%} = \frac{TP * TN - FP * FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} * 100$$

3.2 Adaptive Boosting Model

Table 1: Overall Statistics

Accuracy:	0.6246
95% CI:	(0.617, 0.6322)
No Information Rate:	0.3362
P-Value [Acc > NIR]:	$2.2 * 10^{-16}$
Kappa:	0.5043

Table 2: Statistics by Class

	Sen	Spe	PPV	NPV	Pre	BA	MCC	NoP
cytoplasm	0.53	0.83	0.42	0.89	0.42	0.68	0.33	1557
ER	0.41	0.98	0.45	0.97	0.45	0.69	0.40	283
golgi	0.01	1.00	1.00	0.97	1.00	0.51	0.11	5
lysosome	0.01	1.00	1.00	0.99	1.00	0.50	0.07	1
mitochondrial	0.52	0.97	0.75	0.92	0.75	0.75	0.58	1196
nucleus	0.69	0.87	0.62	0.90	0.62	0.78	0.53	2550
peroxisome	0.00	1.00	NA	0.99	NA	0.50	0.00	0
plasma	0.21	1.00	1.00	1.00	1.00	0.61	0.46	17
secreted	0.79	0.87	0.75	0.89	0.75	0.83	0.65	4187
vacuole	0.56	1.00	1.00	1.00	1.00	0.78	0.75	5

Measures of Key Abbreviations Sen: Sensitivity, Spe: Specificity, PPV: Positive Predicted Value, NPV: Negative Predicted Value, Pre: Prevalence, BA: Balanced Accuracy, MCC: Matthews Correlation Coefficient, NoP: Number of Proteins.

The above ensemble model building was made from 50 trees to test the performance of the final independent model assessment. Table 1. Above summarizes the overall statistics of this model with an overall accuracy of 62% prediction of the proteins subcellular location. Locations with a sensitivity value of 0 and a specificity value of 1 shows that there are no proteins predicted to these subcellular locations.

Also the various statistics by class is presented by Table 2. Above indicated that, vacuole location had the best prediction with an MCC of 75% with its sensitivity and specificity accuracy of 56% and 100% respectively. The sensitivity measure of 56% indicates that the

model correctly predicted the presence of proteins in the vacuole location 56% of the time and the specificity correctly predicted the absence of proteins in this locations 100% of the time. The balanced accuracy of the vacuole location was 78%. The worst prediction of this model was the location of peroxisome which had an MCC of 0% with its sensitivity and specificity of 0% and 100% respectively. Peroxisome location also had a balanced accuracy of 50%.

3.3 Adaptive Boosting Model (Down Sampling)

Table 3: Overall Statistics

Accuracy:	0.3997
95% CI:	(0.3920, 0.4074)
No Information Rate:	0.3362
P-Value [Acc > NIR]:	$3.75 * 10^{-62}$
Kappa:	0.2854

Table 4: Statistics by Class

	Sen	Spe	PPV	NPV	Pre	BA	MCC	NoP
cytoplasm	0.25	0.88	0.33	0.84	0.33	0.57	0.15	746
ER	0.37	0.93	0.19	0.97	0.19	0.65	0.22	253
golgi	0.16	0.94	0.07	0.98	0.06	0.55	0.07	64
lysosome	0.37	0.94	0.07	0.99	0.06	0.65	0.14	67
mitochondrial	0.45	0.95	0.60	0.91	0.11	0.70	0.45	1024
nucleus	0.37	0.93	0.62	0.83	0.14	0.65	0.36	1371
peroxisome	0.47	0.88	0.03	1.00	0.12	0.68	0.09	50
plasma	0.88	0.98	0.18	1.00	0.02	0.93	0.39	70
secreted	0.50	0.91	0.73	0.78	0.23	0.70	0.45	2618
vacuole	0.89	0.98	0.02	1.00	0.03	0.93	0.13	8

Measures of Key Abbreviations Sen: Sensitivity, Spe: Specificity, PPV: Positive Predicted Value, NPV: Negative Predicted Value, Pre: Prevalence, BA: Balanced Accuracy, MCC: Matthews Correlation Coefficient, NoP: Number of Proteins.

From Table 3. above, it can be seen that the overall accuracy of this model is 40% which performed lower than the previous model (Adaptive Boosting without any sampling) in predicting the subcellular locations of proteins.

Table 4. Summarizes the various statistics by class. Above table indicated that, mitochondrial and secreted locations had the best predictions with an MCC of 45% each. The sensitivity accuracy for each predicted locations were 45% and 50% respectively for mitochondrial and secreted which means that the model correctly predicted the presence of proteins 45% and 50% of the time while the specificity were 95% and 91% which means

that the model correctly predicted the absence of proteins 95% and 91% of the time. The balanced accuracies were also 70% for each of the locations. The worst prediction of this model was the location of Golgi apparatus which had an MCC of 7% with its sensitivity and specificity of 16% and 94% respectively. The balanced accuracy of this location was 55%.

3.4 Adaptive Boosting Model (Up Sampling)

Table 5: Overall Statistics

Accuracy:	0.4293
95% CI:	(0.4215, 0.4371)
No Information Rate:	0.3362
P-Value [Acc > NIR]:	$6.80 * 10^{-130}$
Kappa:	0.3282

Table 6: Statistics by Class

	Sen	Spe	PPV	NPV	Pre	BA	MCC	NoP
cytoplasm	0.30	0.87	0.35	0.85	0.35	0.59	0.19	883
ER	0.38	0.94	0.24	0.97	0.24	0.66	0.26	262
golgi	0.20	0.95	0.09	0.98	0.09	0.57	0.10	81
lysosome	0.47	0.94	0.08	0.99	0.08	0.70	0.18	86
mitochondrial	0.38	0.99	0.93	0.90	0.93	0.69	0.56	879
nucleus	0.48	0.91	0.63	0.85	0.63	0.69	0.43	1768
peroxisome	0.60	0.78	0.02	1.00	0.02	0.69	0.08	64
plasma	0.86	1.00	0.87	1.00	0.87	0.93	0.87	69
secreted	0.50	0.97	0.90	0.79	0.90	0.74	0.57	2637
vacuole	0.78	1.00	0.87	1.00	0.88	0.89	0.82	7

Measures of Key Abbreviations Sen: Sensitivity, Spe: Specificity, PPV: Positive Predicted Value, NPV: Negative Predicted Value, Pre: Prevalence, BA: Balanced Accuracy, MCC: Matthews Correlation Coefficient, NoP: Number of Proteins.

The overall accuracy of this model as displayed by Table 5. is 43% even though it was better than the model accuracy of Adaptive Boosting with down sampling, it still fell short to Adaptive Boosting without Sampling

Table 6. above shows the summary statistics by class of the protein locations. Above table revealed that, Plasma membrane location had the optimal prediction with an MCC of 87%. The sensitivity and specificity accuracy of its predicted location was 86% and 100% respectively. The sensitivity measure of 56% indicates that the model correctly predicted the presence of proteins in the vacuole location 86% of the time and the specificity correctly pre-

dicted the absence of proteins in this locations 100% of the time. Furthermore the balanced accuracy that was produced is 93%. The least performing prediction of this model was the peroxisome location which had an MCC of 8% with its sensitivity and specificity being 60% and 78% respectively. The balanced accuracy of this location was 69%.

3.5 Bagging

Table 7: Overall Statistics

Accuracy:	0.8176
95% CI:	(0.8115, 0.8236)
No Information Rate:	0.3362
P-Value [Acc > NIR]:	$62.2 * 10^{-16}$
Kappa:	0.761

Table 8: Statistics by Class

	Sen	Spe	PPV	NPV	Pre	BA	MCC	NoP
cytoplasm	0.75	0.93	0.72	0.94	0.72	0.84	0.67	2202
ER	0.72	0.99	0.82	0.99	0.82	0.86	0.76	499
golgi	0.55	1.00	0.88	0.99	0.88	0.78	0.69	227
lysosome	0.39	1.00	0.96	0.99	0.96	0.69	0.61	71
mitochondrial	0.81	0.98	0.87	0.97	0.87	0.89	0.81	1850
nucleus	0.83	0.94	0.80	0.95	0.80	0.88	0.76	3102
peroxisome	0.29	1.00	0.97	1.00	0.97	0.65	0.53	31
plasma	0.83	1.00	0.99	1.00	0.99	0.91	0.90	66
secreted	0.91	0.92	0.85	0.95	0.85	0.91	0.82	4775
vacuole	0.67	1.00	1.00	1.00	1.00	0.83	0.82	6

Measures of Key Abbreviations Sen: Sensitivity, Spe: Specificity, PPV: Positive Predicted Value, NPV: Negative Predicted Value, Pre: Prevalence, BA: Balanced Accuracy, MCC: Matthews Correlation Coefficient, NoP: Number of Proteins.

The above tables summarize the overall statistics and statistics by each class. Table 7. above indicates the model's overall protein subcellular location of 82% accuracy. Table 8. also illustrates the various statistics by class and shows the assessing of the model. The Plasma membrane location had the best prediction with an MCC of 90% with sensitivity and specificity accuracy of 83% and 100% respectively. The sensitivity measure of 83% indicates that the model correctly predicted the presence of proteins in the plasma membrane location 83% of the time and the specificity accuracy indicates the correctly predicted absence of proteins in this locations 100% of the time. The balanced accuracy of the Plasma

membrane location was 91%. The least prediction of this model was the location of peroxisome which had an MCC of 53% with its sensitivity and specificity of 29% and 100% respectively. Peroxisome membrane location also had a balanced accuracy of 65%.

3.6 Bagging (Down Sampling)

Table 9: Overall Statistics

Accuracy:	0.3782
95% CI:	(0.3706, 0.3858)
No Information Rate:	0.3362
P-Value [Acc > NIR]:	$2.2 * 10^{-16}$
Kappa:	0.2745

Table 10: Statistics by Class

	Sen	Spe	PPV	NPV	Pre	BA	MCC	NoP
cytoplasm	0.17	0.93	0.34	0.83	0.34	0.55	0.15	495
ER	0.41	0.94	0.24	0.97	0.24	0.68	0.31	283
golgi	0.26	0.92	0.08	0.98	0.08	0.59	0.10	107
lysosome	0.41	0.89	0.04	0.99	0.04	0.65	0.11	74
mitochondrial	0.43	0.94	0.53	0.91	0.53	0.68	0.47	980
nucleus	0.48	0.89	0.58	0.85	0.58	0.68	0.42	1778
peroxisome	0.36	0.88	0.02	1.00	0.02	0.62	0.07	38
plasma	0.83	0.99	0.29	1.00	0.29	0.91	0.49	66
secreted	0.40	0.96	0.83	0.76	0.83	0.68	0.46	2104
vacuole	1.00	0.97	0.02	1.00	0.02	0.99	0.15	9

Measures of Key Abbreviations Sen: Sensitivity, Spe: Specificity, PPV: Positive Predicted Value, NPV: Negative Predicted Value, Pre: Prevalence, BA: Balanced Accuracy, MCC: Matthews Correlation Coefficient, NoP: Number of Proteins.

From Table 9. above, it can be seen that the overall accuracy of this model is 38% which performed lower than the previous model (Bagging without any sampling) in predicting the subcellular locations of proteins.

The statistics by class for the various locations has been Summarizes in Table 10 above. This shows that, Plasma membrane location had the optimal prediction with an MCC of 49%. The sensitivity and specificity accuracy of its predicted location was 83% and 99% respectively. The sensitivity measure of 83% indicates that the model correctly predicted the presence of proteins in the vacuole location 83% of the time and the specificity correctly

predicted the absence of proteins in this locations 99% of the time. Also the balanced accuracy that was produced is 91%. The least performing prediction of this model was the peroxisome location which had an MCC of 7% with its sensitivity and specificity being 36% and 88% respectively. The balanced accuracy of this location was 62%.

3.7 Bagging (Up Sampling)

Table 11: Overall Statistics

Accuracy:	0.8069
95% CI:	(0.8006, 0.813)
No Information Rate:	0.3362
P-Value [Acc > NIR]:	$2.2 * 10^{-16}$
Kappa:	0.7495

Table 12: Statistics by Class

	Sen	Spe	PPV	NPV	Pre	BA	MCC	NoP
cytoplasm	0.76	0.92	0.69	0.94	0.69	0.84	0.68	2225
ER	0.77	0.99	0.78	0.99	0.78	0.88	0.77	532
golgi	0.64	1.00	0.78	0.99	0.78	0.82	0.70	262
lysosome	0.47	1.00	0.61	0.99	0.61	0.73	0.52	86
mitochondrial	0.80	0.98	0.86	0.97	0.86	0.89	0.82	1837
nucleus	0.82	0.93	0.80	0.94	0.80	0.88	0.77	3060
peroxisome	0.44	1.00	0.46	1.00	0.46	0.72	0.47	47
plasma	0.85	1.00	0.92	1.00	0.92	0.92	0.89	68
secreted	0.86	0.94	0.88	0.93	0.88	0.90	0.81	4537
vacuole	0.78	1.00	0.78	1.00	0.78	0.89	0.78	7

Measures of Key Abbreviations Sen: Sensitivity, Spe: Specificity, PPV: Positive Predicted Value, NPV: Negative Predicted Value, Pre: Prevalence, BA: Balanced Accuracy, MCC: Matthews Correlation Coefficient, NoP: Number of Proteins.

Table 11. displays the overall accuracy of this model with 81% even though it was better than the model accuracy of Bagging with down sampling, it still fell short to Bagging without Sampling.

Table 12. above indicates the summary statistics by class of the protein locations. Above table revealed that, Plasma membrane location had the optimal prediction with an MCC of 89%. The sensitivity and specificity accuracy of its predicted location was 85% and 100% respectively. The sensitivity measure of 85% indicates that the model correctly predicted the presence of proteins in the plasma location 85% of the time and the specificity correctly

predicted the absence of proteins in this locations 100% of the time. Furthermore the balanced accuracy that was produced is 95%. The least performing prediction of this model was the peroxisome location which had an MCC of 47% with its sensitivity and specificity being 44% and 100% respectively. The balanced accuracy of this location was 72%.

3.8 Cross Validated Bagging

Table 13: Overall Statistics

Accuracy:	0.8188
95% CI:	(0.8127, 0.8248)
No Information Rate:	0.3362
P-Value [Acc > NIR]:	$2.2 * 10^{-16}$
Kappa:	0.7625

Table 14: Statistics by Class

	Sen	Spe	PPV	NPV	Pre	BA	MCC	NoP
cytoplasm	0.74	0.94	0.73	0.94	0.73	0.84	0.67	2181
ER	0.72	0.99	0.83	0.99	0.83	0.86	0.76	498
golgi	0.53	1.00	0.90	0.99	0.90	0.76	0.69	217
lysosome	0.45	1.00	0.98	0.99	0.98	0.73	0.66	82
mitochondrial	0.81	0.98	0.88	0.97	0.88	0.89	0.82	1847
nucleus	0.85	0.93	0.80	0.95	0.80	0.89	0.76	3148
peroxisome	0.39	1.00	0.95	1.00	0.95	0.69	0.61	41
plasma	0.79	1.00	0.97	1.00	0.97	0.89	0.87	63
secreted	0.90	0.92	0.85	0.95	0.85	0.91	0.81	4765
vacuole	0.67	1.00	0.86	1.00	0.86	0.83	0.76	6

Measures of Key Abbreviations Sen: Sensitivity, Spe: Specificity, PPV: Positive Predicted Value, NPV: Negative Predicted Value, Pre: Prevalence, BA: Balanced Accuracy, MCC: Matthews Correlation Coefficient, NoP: Number of Proteins.

The above tables summarize the overall statistics and statistics by each class. Table 13. above indicates the model's overall protein subcellular location of 87% accuracy. Table 14. also illustrates the various statistics by class and shows the assessing of the model. The Plasma membrane location had the best prediction with an MCC of 87% with sensitivity and specificity accuracy of 79% and 100% respectively. The sensitivity measure of 79% indicates that the model correctly predicted the presence of proteins in the plasma membrane location 79% of the time and the specificity accuracy indicates the correctly predicted absence of proteins in this locations 100% of the time. The balanced accuracy of the Plasma

membrane location was 89%. The least prediction of this model was the location of peroxisome which had an MCC of 61% with its sensitivity and specificity of 39% and 100% respectively. Peroxisome membrane location also had a balanced accuracy of 69%.

3.9 Cross Validated Bagging (Down Sampling)

Table 15: Overall Statistics

Accuracy:	0.3782
95% CI:	(0.8127, 0.8248)
No Information Rate:	0.3362
P-Value [Acc > NIR]:	$2.2 * 10^{-16}$
Kappa:	0.2745

Table 16: Statistics by Class

	Sen	Spe	PPV	NPV	Pre	BA	MCC	NoP
cytoplasm	0.17	0.93	0.34	0.83	0.34	0.55	0.13	495
ER	0.41	0.94	0.24	0.97	0.24	0.68	0.27	283
golgi	0.26	0.92	0.08	0.98	0.08	0.59	0.11	107
lysosome	0.41	0.89	0.04	0.99	0.04	0.65	0.10	74
mitochondrial	0.43	0.94	0.53	0.91	0.53	0.68	0.40	980
nucleus	0.48	0.89	0.58	0.85	0.58	0.68	0.39	1778
peroxisome	0.36	0.88	0.02	1.00	0.02	0.62	0.06	38
plasma	0.83	0.99	0.29	1.00	0.29	0.91	0.49	66
secreted	0.40	0.96	0.83	0.76	0.83	0.68	0.46	2104
vacuole	1.00	0.97	0.02	1.00	0.02	0.99	0.14	9

Measures of Key Abbreviations Sen: Sensitivity, Spe: Specificity, PPV: Positive Predicted Value, NPV: Negative Predicted Value, Pre: Prevalence, BA: Balanced Accuracy, MCC: Matthews Correlation Coefficient, NoP: Number of Proteins.

Table 15. above displays the overall accuracy of 38% which performed lower than the previous model (Cross Validated Bagging without any sampling) in predicting the subcellular locations of proteins.

The statistics by class for the various locations has been Summarizes in Table 16 above. This shows that, Plasma membrane location had the optimal prediction with an MCC of 49%. The sensitivity and specificity accuracy of its predicted location was 83% and 99% respectively. The sensitivity measure of 83% indicates that the model correctly predicted the presence of proteins in the vacuole location 83% of the time and the specificity correctly

predicted the absence of proteins in this locations 99% of the time. Also the balanced accuracy that was produced is 91%. The least performing prediction of this model was the peroxisome location which had an MCC of 6% with its sensitivity and specificity being 36% and 88% respectively. The balanced accuracy of this location was 62%.

3.10 Cross Validated Bagging (Up Sampling)

Table 17: Overall Statistics

Accuracy:	0.8069
95% CI:	(0.8006, 0.813)
No Information Rate:	0.3362
P-Value [Acc > NIR]:	$2.2 * 10^{-16}$
Kappa:	0.74955

Table 18: Statistics by Class

	Sen	Spe	PPV	NPV	Pre	BA	MCC	NoP
cytoplasm	0.76	0.92	0.69	0.94	0.69	0.84	0.66	2225
ER	0.77	0.99	0.78	0.99	0.78	0.88	0.76	532
golgi	0.64	1.00	0.78	0.99	0.78	0.82	0.7	262
lysosome	0.47	1.00	0.61	0.99	0.61	0.73	0.53	86
mitochondrial	0.80	0.98	0.86	0.97	0.86	0.89	0.8	1837
nucleus	0.82	0.93	0.80	0.94	0.80	0.88	0.75	3060
peroxisome	0.44	1.00	0.46	1.00	0.46	0.72	0.45	47
plasma	0.85	1.00	0.92	1.00	0.92	0.92	0.88	68
secreted	0.86	0.94	0.88	0.93	0.88	0.90	0.8	4537
vacuole	0.78	1.00	0.78	1.00	0.78	0.89	0.78	7

Measures of Key Abbreviations Sen: Sensitivity, Spe: Specificity, PPV: Positive Predicted Value, NPV: Negative Predicted Value, Pre: Prevalence, BA: Balanced Accuracy, MCC: Matthews Correlation Coefficient, NoP: Number of Proteins.

Table 19 and Table 20 depicts the summary results for Random Forest model. Table 19 shows that the model can predict proteins to their subcellular location with a 85% accuracy. Table 20 illustrates statistics by class and indicates the performance of the model using the testing data.

The plasma membrane location has the highest MCC value of 96%, with a sensitivity of 93% and a specificity of 100%. This location has a balanced accuracy of 96%. The least predicted location of this model was the peroxisome subcellular location which has an MCC value of 60%. The sensitivity, specificity, and balanced accuracy values for the peroxisome are 37%,

100%, and 68%.

3.11 Random Forest

Table 19: Overall Statistics

Accuracy:	0.8548
95% CI:	(0.8491, 0.8602)
No Information Rate:	0.3362
P-Value [Acc > NIR]:	$2.2 * 10^{-16}$
Kappa:	0.8092

Table 20: Statistics by Class

	Sen	Spe	PPV	NPV	Pre	BA	MCC	NoP
cytoplasm	0.79	0.95	0.78	0.95	0.78	0.87	0.73	2313
ER	0.77	1.00	0.89	0.99	0.89	0.88	0.82	535
golgi	0.57	1.00	0.98	0.99	0.98	0.78	0.74	233
lysosome	0.45	1.00	1.00	0.99	1.00	0.72	0.66	81
mitochondrial	0.82	0.99	0.95	0.97	0.95	0.91	0.86	1887
nucleus	0.88	0.95	0.83	0.96	0.83	0.91	0.81	3276
peroxisome	0.37	1.00	0.97	1.00	0.98	0.68	0.60	39
plasma	0.93	1.00	0.99	1.00	0.99	0.96	0.96	74
secreted	0.94	0.93	0.86	0.97	0.86	0.93	0.85	4967
vacuole	0.78	1.00	1.00	1.00	1.00	0.89	0.88	7

Measures of Key Abbreviations Sen: Sensitivity, Spe: Specificity, PPV: Positive Predicted Value, NPV: Negative Predicted Value, Pre: Prevalence, BA: Balanced Accuracy, MCC: Matthews Correlation Coefficient, NoP: Number of Proteins.

The overall accuracy of this model as displayed by Table 17. is 81% which fell short to cross validation with Bagging without Sampling.

Table 18. above shows the summary statistics by class of the protein locations. Above table indicates that, Plasma membrane location had the optimal prediction with an MCC of 49%. The sensitivity and specificity accuracy of its predicted location was 83% and 99% respectively .The sensitivity measure of 83% indicates that the model correctly predicted the presence of proteins in the plasma location 83% of the time and the specificity correctly predicted the absence of proteins in this locations 99% of the time. Furthermore the balanced

accuracy that was produced is 91%. The least performing prediction of this model was the peroxisome location which had an MCC of 6% with its sensitivity and specificity being 36% and 88% respectively. The balanced accuracy of this location was 62%.

3.12 Random Forest (Down Sampling)

Table 21: Overall Statistics

Accuracy:	0.4455
95% CI:	(0.4377, 0.4534)
No Information Rate:	0.3362
P-Value [Acc > NIR]:	$2.01 * 10^{-117}$
Kappa:	0.3364

Table 22: Statistics by Class

	Sen	Spe	PPV	NPV	Pre	BA	MCC	NoP
cytoplasm	0.32	0.87	0.36	0.85	0.36	0.60	0.20	951
ER	0.54	0.93	0.26	0.98	0.26	0.74	0.34	375
golgi	0.24	0.94	0.10	0.98	0.10	0.59	0.12	99
lysosome	0.30	0.93	0.05	0.99	0.05	0.62	0.10	54
mitochondrial	0.46	0.97	0.71	0.91	0.71	0.71	0.52	1047
nucleus	0.44	0.91	0.59	0.84	0.59	0.67	0.38	1630
peroxisome	0.57	0.90	0.04	1.00	0.04	0.73	0.12	60
plasma	1.00	0.99	0.34	1.00	0.34	0.99	0.58	80
secreted	0.51	0.94	0.82	0.79	0.82	0.73	0.52	2688
vacuole	0.78	0.99	0.05	1.00	0.05	0.88	0.19	7

Measures of Key Abbreviations Sen: Sensitivity, Spe: Specificity, PPV: Positive Predicted Value, NPV: Negative Predicted Value, Pre: Prevalence, BA: Balanced Accuracy, MCC: Matthews Correlation Coefficient, NoP: Number of Proteins.

From Table 21. above, it can be seen that the overall accuracy of this model is 45% which performed lower than the previous model (Random Forest without any sampling) in predicting the subcellular locations of proteins.

The statistics by class for the various locations has been Summarize in Table 22 above. This shows that, Plasma membrane location had the optimal prediction with an MCC of 58%. The sensitivity and specificity accuracy of its predicted location was 100% and 99% respectively. The sensitivity measure of 100% indicates that the model correctly predicted the presence of proteins in the vacuole location 100% of the time and the specificity correctly predicted

the absence of proteins in this locations 99% of the time. Also the balanced accuracy that was produced is 99%. The least performing prediction of this model was the peroxisome location which had an MCC of 12% with its sensitivity and specificity being 57% and 90% respectively. The balanced accuracy of this location was 73%.

3.13 Random Forest (Up Sampling)

Table 23: Overall Statistics

Accuracy:	0.8618
95% CI:	(0.8563, 0.8672)
No Information Rate:	0.3362
P-Value [Acc > NIR]:	$2.2 * 10^{-16}$
Kappa:	0.819

Table 24: Statistics by Class

	Sen	Spe	PPV	NPV	Pre	BA	MCC	NoP
cytoplasm	0.81	0.94	0.76	0.96	0.76	0.88	0.73	2366
ER	0.82	1.00	0.89	0.99	0.89	0.91	0.85	569
golgi	0.63	1.00	0.99	0.99	0.99	0.82	0.79	259
lysosome	0.51	1.00	1.00	0.99	1.00	0.75	0.71	92
mitochondrial	0.82	1.00	0.97	0.97	0.97	0.91	0.88	1876
nucleus	0.88	0.95	0.84	0.96	0.84	0.91	0.82	3285
peroxisome	0.45	1.00	0.98	1.00	0.98	0.73	0.66	48
plasma	0.99	1.00	1.00	1.00	1.00	0.99	0.99	79
secreted	0.94	0.94	0.88	0.97	0.88	0.94	0.86	4942
vacuole	0.78	1.00	1.00	1.00	1.00	0.89	0.88	7

Measures of Key Abbreviations Sen: Sensitivity, Spe: Specificity, PPV: Positive Predicted Value, NPV: Negative Predicted Value, Pre: Prevalence, BA: Balanced Accuracy, MCC: Matthews Correlation Coefficient, NoP: Number of Proteins.

Table 23. displays the overall accuracy of this model with 86% which is better than the accuracy of the above two models.

Table 24. above indicates the summary statistics by class of the protein locations. Above table revealed that, Plasma membrane location had the highest prediction with an MCC of 99%. The sensitivity and specificity accuracy of its predicted location was 99% and 100% respectively. The sensitivity measure of 99% indicates that the model correctly predicted the presence of proteins in the plasma location 99% of the time and the specificity correctly predicted the absence of proteins in this locations 100% of the time. The balanced accuracy

that was produced is 99%. The least performing prediction of this model was the peroxisome location which had an MCC of 66% with its sensitivity and specificity being 45% and 100% respectively. The balanced accuracy of this location was 73%.

3.14 XGBoost

Table 25: Overall Statistics

Accuracy:	0.7436
95% CI:	(0.7367, 0.7504)
No Information Rate:	0.3362
P-Value [Acc > NIR]:	$2.2 * 10^{-16}$
Kappa:	0.6647

Table 26: Statistics by Class

	Sen	Spe	PPV	NPV	Pre	BA	MCC	NoP
cytoplasm	0.62	0.90	0.58	0.91	0.58	0.76	0.50	1807
ER	0.64	0.98	0.66	0.98	0.66	0.81	0.63	439
golgi	0.34	1.00	0.70	0.98	0.70	0.67	0.48	141
lysosome	0.26	1.00	0.68	0.99	0.68	0.63	0.42	47
mitochondrial	0.75	0.97	0.80	0.96	0.80	0.86	0.74	1722
nucleus	0.77	0.91	0.73	0.93	0.73	0.84	0.67	2860
peroxisome	0.24	1.00	0.96	0.99	0.96	0.62	0.47	25
plasma	0.95	1.00	1.00	1.00	1.00	0.98	0.97	76
secreted	0.86	0.91	0.83	0.93	0.83	0.89	0.77	4544
vacuole	0.78	1.00	1.00	1.00	1.00	0.89	0.88	7

Measures of Key Abbreviations Sen: Sensitivity, Spe: Specificity, PPV: Positive Predicted Value, NPV: Negative Predicted Value, Pre: Prevalence, BA: Balanced Accuracy, MCC: Matthews Correlation Coefficient, NoP: Number of Proteins.

The 5-fold cross validation was used to train this model. The following tables summarizes the outcome of testing the final model. Table 25 and Table 26 shows the summary results for XGBoost model. Table 25 shows that the model can predict proteins to their subcellular location with a 74% accuracy. Table 26 illustrates statistics by class and indicates the performance of the model using the testing data.

The plasma membrane location has the highest MCC value of 97%, with a sensitivity of 95% and a specificity of 100%. This location has a balanced accuracy of 98%. The least predicted location of this model was the lysosome subcellular location which has MCC value of 42%.

The sensitivity, specificity, and balanced accuracy values for the lysosome are 26%, 100%, and 63% .

3.15 XGBoost (Down Sampling)

Table 27: Overall Statistics

Accuracy:	0.3153
95% CI:	(0.3080, 0.3226)
No Information Rate:	0.3362
P-Value [Acc > NIR]:	0.9999
Kappa:	0.6983

Table 28: Statistics by Class

	Sen	Spe	PPV	NPV	Pre	BA	MCC	NoP
cytoplasm	0.32	0.84	0.31	0.84	0.19	0.58	0.16	936
ER	0.29	0.90	0.12	0.97	0.04	0.60	0.13	201
golgi	0.25	0.93	0.08	0.98	0.03	0.59	0.10	101
lysosome	0.38	0.90	0.04	0.99	0.01	0.64	0.10	69
mitochondrial	0.28	0.93	0.39	0.88	0.15	0.60	0.24	640
nucleus	0.36	0.91	0.56	0.82	0.24	0.63	0.32	1331
peroxisome	0.35	0.90	0.02	1.00	0.01	0.63	0.07	37
plasma	1.00	0.99	0.48	1.00	0.01	1.00	0.69	80
secreted	0.29	0.92	0.66	0.72	0.34	0.61	0.28	1544
vacuole	0.89	0.99	0.05	1.00	0.00	0.94	0.22	8

Measures of Key Abbreviations Sen: Sensitivity, Spe: Specificity, PPV: Positive Predicted Value, NPV: Negative Predicted Value, Pre: Prevalence, BA: Balanced Accuracy, MCC: Matthews Correlation Coefficient, NoP: Number of Proteins.

The 5-fold cross validation was trained and down sample most of the class. From Table 27. above, it can be seen that the overall accuracy of this model is 32% which performed lower than the previous model (XGBoost without any sampling) in predicting the subcellular locations of proteins.

The statistics by class for the various locations has been Summarizes in Table 28 above. This shows that, Plasma membrane location had the optimal prediction with an MCC of 69%. The sensitivity and specificity accuracy of its predicted location was 100% and 99% respectively. The sensitivity measure of 100% indicates that the model correctly predicted the presence of

proteins in this location 100% of the time and the specificity correctly predicted the absence of proteins in this locations 99% of the time. Also the balanced accuracy that was produced is 100%. The least performing prediction of this model were the peroxisome locations with MCC of both 7%.

3.16 XGBoost (Up Sampling)

Table 29: Overall Statistics

Accuracy:	0.7129
95% CI:	(0.7057, 0.7200)
No Information Rate:	0.3362
P-Value [Acc > NIR]:	0.0348
Kappa:	0.6369

Table 30: Statistics by Class

	Sen	Spe	PPV	NPV	Pre	BA	MCC	NoP
cytoplasm	0.58	0.90	0.57	0.90	0.57	0.74	0.47	1685
ER	0.74	0.97	0.52	0.99	0.52	0.86	0.60	514
golgi	0.68	0.96	0.34	0.99	0.34	0.82	0.46	277
lysosome	0.71	0.98	0.31	1.00	0.31	0.85	0.47	130
mitochondrial	0.75	0.96	0.78	0.96	0.78	0.86	0.73	1720
nucleus	0.72	0.93	0.76	0.91	0.76	0.82	0.66	2664
peroxisome	0.59	0.99	0.26	1.00	0.26	0.79	0.39	63
plasma	1.00	1.00	1.00	1.00	1.00	1.00	1.00	80
secreted	0.77	0.96	0.90	0.89	0.90	0.86	0.76	4045
vacuole	0.89	1.00	1.00	1.00	1.00	0.94	0.94	8

Measures of Key Abbreviations Sen: Sensitivity, Spe: Specificity, PPV: Positive Predicted Value, NPV: Negative Predicted Value, Pre: Prevalence, BA: Balanced Accuracy, MCC: Matthews Correlation Coefficient, NoP: Number of Proteins.

The overall accuracy of this model as displayed by Table 29. is 71% . The locations with a sensitivity value of 0 and a specificity value of 1 indicate that there are no proteins predicted to these subcellular locations.

Table 30. On the other hand also shows the summary statistics by class of the protein locations. Vacuole location recorded the highest prediction with an MCC of 75%. The sensitivity and specificity accuracy of its predicted location was 56% and 100% respectively .The sensitivity measure of 56% indicates that the model correctly predicted the presence of proteins in this location 56% of the time and the specificity correctly predicted the absence

of proteins in this locations 100% of the time. Furthermore the balanced accuracy that was produced is 78%. The next best performing prediction of this model was the secreted location which has an MCC of 65% with its sensitivity and specificity being 79% and 87% respectively. The balanced accuracy of this location was 83%.

3.17 Deep Neural Network

Table 31: Overall Statistics

Accuracy:	0.7674
Kappa:	0.701

Table 32: Statistics by Class

	Sen	Spe	PPV	NPV	Pre	BA	MCC	NoP
cytoplasm	0.33	1.00	0.93	0.05	0.96	0.67	0.55	1685
ER	0.92	0.59	0.92	3.50	0.50	0.76	0.58	514
golgi	0.57	0.99	0.76	0.52	0.71	0.78	0.63	277
lysosome	0.70	0.99	0.71	1.11	0.51	0.85	0.59	130
mitochondrial	0.85	0.97	0.97	0.83	0.87	0.91	0.83	1720
nucleus	0.64	0.98	0.96	0.20	0.92	0.81	0.72	2664
peroxisome	0.31	0.99	0.75	0.15	0.87	0.65	0.52	63
plasma	1.00	1.00	1.00	N/A	1.00	1.00	1.00	80
secreted	0.81	0.98	0.99	0.23	0.96	0.90	0.83	4045
vacuole	0.89	1.00	1.00	0.00	1.00	0.94	0.94	8

Measures of Key Abbreviations Sen: Sensitivity, Spe: Specificity, PPV: Positive Predicted Value, NPV: Negative Predicted Value, Pre: Prevalence, BA: Balanced Accuracy, MCC: Matthews Correlation Coefficient, NoP: Number of Proteins.

Table 31 and Table 32 shows the summary results for Deep Neural Network model. Table 31 shows that the model can predict proteins to their subcellular location with a 77% accuracy. Table 32 shows the performance of the model using the testing data.

The plasma membrane location has the highest MCC value of 100%, with a sensitivity of 100% and specificity of 100%. The sensitivity measure of 100% indicates that the model perfectly predicted the presence of proteins in the plasma location and the specificity correctly predicted a perfect absence of proteins in this location. Furthermore the balanced accuracy that was produced is 100%. The least performing prediction of this model was the cytoplasm location which had an MCC of 55% with its sensitivity and specificity being 33% and 100% respectively. The balanced accuracy of this location was 67%.

3.18 Automatic Machine Learning (AutoML)

Table 33: Overall Statistics

Accuracy:	0.8987
Kappa:	0.868

Table 34: Statistics by Class

	Sen	Spe	PPV	NPV	Pre	BA	MCC	NoP
cytoplasm	0.81	1.00	1.00	0.05	0.99	0.91	0.89	792
ER	0.98	0.93	0.99	0.93	0.77	0.95	0.83	3001
golgi	0.50	1.00	0.92	0.14	0.93	0.75	0.67	201
lysosome	0.52	1.00	0.77	0.41	0.75	0.76	0.63	89
mitochondrial	0.99	0.98	1.00	0.88	0.95	0.99	0.56	3725
nucleus	0.51	1.00	1.00	0.00	1.00	0.76	0.69	981
peroxisome	0.58	1.00	1.00	0.00	1.00	0.79	0.76	88
plasma	1.00	1.00	1.00	N/A	1.00	1.00	1.00	113
secreted	0.99	0.96	1.00	0.93	0.92	0.98	0.93	5103
vacuole	1.00	1.00	1.00	N/A	1.00	1.00	1.00	9

Measures of Key Abbreviations Sen: Sensitivity, Spe: Specificity, PPV: Positive Predicted Value, NPV: Negative Predicted Value, Pre: Prevalence, BA: Balanced Accuracy, MCC: Matthews Correlation Coefficient, NoP: Number of Proteins.

The above tables summarize the overall statistics and statistics by each class. Table 33. above indicates the model's overall protein subcellular location of 90% accuracy.

Table 34. Also illustrates the various statistics by class and shows the assessing of the model. The Plasma membrane location had the best prediction with an MCC of 100% with sensitivity and specificity accuracy of 100% and 100% respectively. The sensitivity measure of 100% indicates that the model correctly predicted the presence of proteins in the plasma membrane location 100% of the time and the specificity accuracy indicates the perfect prediction of absence of proteins in this locations. The balanced accuracy of the Plasma membrane location was 100%. The least prediction of this model was the location of

mitochondrial location which had an MCC of 56% with its sensitivity and specificity of 99% and 98% respectively. The mitochondrial location also had a balanced accuracy of 99%.

Table 35: Comparison of best 3 prediction accuracies of animal protein subcellular location by different algorithms

Subcellular Locations	Predictors	Sen	Spe	BA	MCC
cytoplasm	Automatic Machine Learning	0.81	1.00	0.95	0.89
	Random Forest with Up Sampling	0.81	0.94	0.88	0.73
	Random Forest	0.79	0.95	0.87	0.73
ER	Random Forest with Up Sampling	0.82	1.00	0.91	0.85
	Automatic Machine Learning	0.98	0.93	0.95	0.83
	Random Forest	0.77	1.00	0.88	0.82
golgi	Random Forest with Up Sampling	0.63	1.00	0.82	0.79
	Random Forest	0.57	1.00	0.78	0.74
	Bagging with Up Sampling	0.64	1.00	0.82	0.70
lysosome	Random Forest with Up Sampling	0.51	1.00	0.75	0.71
	Bagging with CV	0.45	1.00	0.73	0.66
	Random Forest	0.45	1.00	0.72	0.66
mitochondrial	Random Forest with Up Sampling	0.82	1.00	0.91	0.88
	Random Forest	0.82	0.99	0.91	0.86
	Bagging with Up Sampling	0.80	0.98	0.89	0.82
nucleus	Random Forest with Up Sampling	0.88	0.95	0.91	0.82
	Random Forest	0.88	0.95	0.91	0.81
	Bagging with Up Sampling	0.82	0.93	0.88	0.77
peroxisome	Automatic Machine Learning	0.58	1.00	0.79	0.76
	Random Forest with Up Sampling	0.45	1.00	0.73	0.66
	Bagging with CV	0.39	1.00	0.69	0.61
plasma	XGBoosT with Up Sampling	1.00	1.00	1.00	1.00
	Automatic Machine Learning	1.00	1.00	1.00	1.00
	Random Forest with Up Sampling	0.99	1.00	0.99	0.99
secreted	Automatic Machine Learning	0.99	0.96	0.98	0.93
	Random Forest with Up Sampling	0.94	0.94	0.94	0.86
	Random Forest	0.94	0.93	0.93	0.85
vacuole	Automatic Machine Learning	1.00	1.00	1.00	1.00
	XGBoosT with Up Sampling	0.89	1.00	0.94	0.94
	Random Forest with Up Sampling	0.78	1.00	0.89	0.88

Table 36: Best Models For Each Subcellular Location

Subcellular Locations	Predictors	Sen	Spe	PPV	NPV	BA	MCC
cytoplasm	Automatic Machine Learning	0.81	1.00	1.00	0.05	0.91	0.89
ER	Random Forest with Up Sampling	0.82	1.00	0.89	0.99	0.91	0.85
golgi	Random Forest with Up Sampling	0.63	1.00	0.99	0.99	0.82	0.79
lysosome	Random Forest with Up Sampling	0.51	1.00	1.00	0.99	0.75	0.71
mitochondrial	Random Forest with Up Sampling	0.82	1.00	0.97	0.97	0.91	0.88
nucleus	Random Forest with Up Sampling	0.88	0.95	0.84	0.96	0.91	0.82
peroxisome	Automatic Machine Learning	0.58	1.00	1.00	0.00	0.79	0.76
plasma	AutoML/XGBoost with Up Sampling	1.00	1.00	1.00	1.00	1.00	1.00
secreted	Automatic Machine Learning	.99	0.96	1.00	0.00	0.98	0.93
vacuole	Automatic Machine Learning	1.00	1.00	1.00	N/A	1.00	1.00

4 Discussion

Several prediction techniques such as Random Forest, Bagging, Stage-wise Addictive Model using Multi-class Exponential loss function (SAMME), Artificial Neural Network, Support Vector Machines (SVMs), Adaptive Boosting and many more have been employed to predict the subcellular location in the research industry. This study focuses on seven of these prediction techniques. Every technique employed in this study comes with its own merits and demerits. The decision for selection rely on the various characteristics that distinguish positive and negative of the data points as well as how the training and testing data set presents itself. In building the model for the prediction of subcellular location of animal proteins, the Pseudo Amino Acid composition was employed in this study.

Adaboost (Adaptive Boosting), Bagging, Cross Validation Bagging, Random Forest, XGBoost, Deep Neural Network and the Automatic Machine learning technique were used to create the various models for predictions. The resampling methods were also incorporated in the first five of these modeling techniques to achieve our objective of the study. The pseudo amino acid composition was employed for the prediction of proteins to their subcellular locations using Matthew's correlation coefficient (MCC), Sensitivity and Specificity of each subcellular location were also used as classification parameters to achieved the aim of the study. Random Forest with Up Sampling algorithm recorded the highest MCC for seven(5) of subcellular location. These includes Endoplasmic Reticulum (ER), Golgi, Lysosome, Mitochondrial and Nucleus s subcellular locations. The Automatic Machine learning algorithm on the other hand demonstrated its best predictions in cytoplasm, peroxisome, secreted and vacuole but had a best joint accuracy prediction with XGBoost with up sampling in plasma membrane.

From table 35 above, Automated Machine learning and XGBoost predicted the plasma membrane location with an optimal MCC whereas random forest with up sampling produced a decreased MCC of 99% in predicting plasma location. Cytoplasm had AutoML as its best

predictor with MCC of 89%. The MCC of cytoplasm again had a moderate prediction of 73% for both Random Forest and with sampling. Although Random Forest and with up sampling predicted moderate MCCs for these location, the model can still be improve to attain it optimal. Predicting ER, Mitochondrial and Nucleus with Random Forest with up sampling had all MCC of above 80% which was an improvement compared to lysosome and Golgi. The MCCs of Lysosome and peroxisome was fairly moderate compared to the other subcellular location predicted by Random Forest. Furthermore bagging with up sampling moderately predicted golgi fairly with MCC of 70% accuracy. This model improved its prediction accuracy for Nucleus with MCCs of 77% and even did better with a higher prediction in the mitochondrial location with MCC of 82% accuracy. Lysosome and peroxisome subcellular locations were also predicted fairly by Bagging with Cross Validation. The prediction accuracy for these locations recorded a MCC of 66% and 61% respectively. Although the locations are accurately predicted fairly by this model, it can still do better with some improvement to it. The secreted location was highly predicted by the Random forest model with 85% MCC accuracy which was even the least among the predictors of the secreted subcellular locations. Similarly Plasma location also had a very high prediction accuracy of 99% MCC and was the third highest by the Random forest with up sampling model.

The XGBoost with up sampling and the Automatic machine learning models jointly had the highest prediction to the plasma location with a perfect MCC of 100% each followed by the vacuole location which had MCC of 94% with the XGBoost with up sampling model. Furthermore Random Forest with up sampling had the best protein prediction among five subcellular locations with mitochondrial location having the highest MCC of 88%. ER location was the second best prediction with MCC of 85%. The prediction accuracy of Random Forest with Up sampling continued to decrease across the other three subcellular location with Peroxisome having the lowest with this model with MCC of 66%. This prediction accuracy can still be improved to provide a better results to this location.

Table 36 above summarizes the best models for each subcellular location. Random Forest with up sampling and the automatic machine learning appears to dominate in predicting the highest accuracy among the various subcellular locations. Mitochondrial recorded the best performance with this method whereas lysosome had the least prediction accuracy. Proteins were perfectly predicted by XGBoost with up sampling and the automatic machine learning to the plasma subcellular locations. The automatic machine had a perfect protein prediction to the vacuole locations too.

5 Conclusion

The above study has clearly shown a progressive accuracy prediction of proteins in their respective subcellular locations as compared to past studies. The various algorithms that were used in the study had a significant impact in predicting the subcellular location of animal proteins. These algorithms were Adaptive Boosting, Bagging, Cross validated Bagging, Random Forest, Xgboost, Deep neural Network and the Automatic Machine learning. In addition, the up and down sampling methods of these models were also employed in this study. The poor performance of most down sampling algorithms employed in this study was as a result of reasons outlined in previous literatures. In general, algorithms with up sampling did better than those with down sampling without resampling.

The study critically revealed that, Random Forest with up sampling and the Automatic machine learning dominated the prediction of proteins subcellular locations accurately to perfection. The accuracy of proteins to these locations were made with high precision and Sensitivity values. The MCC,s of these subcellular locations were also close to perfection which indicated that the actual data has a high correlation with the prediction accuracy.

Furthermore the Xgboost with up sampling and the automatic machine learning algorithms did better than all the other models with the highest precision and MCC. The algorithms were able to produce a perfect prediction of proteins to the plasma membrane locations with a very high precision and sensitivity value. The automatic machine learning also had a perfect accuracy, sensitivity and prediction in identifying proteins to the vacuole locations.

In conclusion, future research work can still be embark to improve the prediction accuracy of the other subcellular location that are to perfections. Algorithms such as Stacking which is also an ensemble method and convolutional neural networks can be applied to improve these prediction accuracies.

\newpage

```

\section{Appendix}

\subsubsection{R codes}

library(seqinr)
library(protr)
library(party)
library(BioSeqClass)
library(randomForest)
library(lme4)
library(caret)
library(adabag)
library(tree)
library(rpart)
library(DMwR)


\#modification to featurepseudoAAComp because of problems
myfeaturePseudoAAComp=function(seq,d,w=.05){
  H1 = c(0.62, -2.53, -0.78, -0.09, 0.29, -0.85, -0.74, 0.48,
        -0.4, 1.38, 1.53, -1.5, 0.64, 1.19, 0.12, -0.18, -0.05,
        0.81, 0.26, 1.8)
  names(H1) = c("A", "R", "N", "D", "C", "Q", "E", "G", "H",
               "I", "L", "K", "M", "F", "P", "S", "T", "W", "Y", "V")
  H2 = c(-0.5, 3, 0.2, 3, -1, 0.2, 3, 0, -0.5, -1.8, -1.8,
        3, -1.3, -2.5, 0, 0.3, -0.4, -3.4, -2.3, -1.5)
  names(H2) = c("A", "R", "N", "D", "C", "Q", "E", "G", "H",
               "I", "L", "K", "M", "F", "P", "S", "T", "W", "Y", "V")
  M = c(71.03711, 156.10111, 114.04293, 115.02694, 103.00919,

```



```

128.05858, 129.04259, 57.02146, 137.05891, 113.08406,
113.08406, 128.09496, 131.04049, 147.06841, 97.05276, 87.03203, 101.04768, 186.07
names(M) = c("A", "R", "N", "D", "C", "Q", "E", "G", "H",
            "I", "L", "K", "M", "F", "P", "S", "T", "W", "Y", "V")
H1 = (H1 - mean(H1))/sd(H1)
H2 = (H2 - mean(H2))/sd(H2)
M = (M - mean(M))/sd(M)
R <- function(aa1, aa2) {
  ((H1[aa1] - H1[aa2])^2 + (H2[aa1] - H2[aa2])^2 + (M[aa1] -
                                                    M[aa2])^2)/3
}

compo = featureFragmentComposition(seq, 1, elements("aminoacid"))
pseudo = sapply(seq, function(x) {
  y = unlist(strsplit(x, split = ""))
  nTotal = length(y)
  sapply(1:d, function(i) {
    mean(sapply(1:(nTotal - i), function(j) {
      R(y[j], y[j + i])
    })))
  })
})

# CODE TO FIX THE PROBLEMS
protlist=lapply(X=1:length(data),FUN=function(z){
  a=unlist(compo[1,z])
  b=pseudo[,z]
  b=b/sum(b)

```

```

    c=c(a,b)
    c=c/sum(c)
    return(c)
})

# making data frame with each row being a protein
paa=data.frame()
for(i in 1:length(data)){
  paa=rbind(paa,protlist[[i]])
}

# fixing row names and column names
rownames(paa)=names(seq)
colnames(paa)=c("PAC:R", "PAC:K", "PAC:E", "PAC:D", "PAC:Q", "PAC:N", "PAC:W", "PAC:G",
                "PAC:S", "PAC:T", "PAC:P", "PAC:H", "PAC:Y", "PAC:C", "PAC:V", "PAC:L")
return(paa)
}

```

```

data = readFASTA("C:/Users/btannan/Desktop/Proteinclean/cyto.fas")\#,seqtype= "AA", as.s
PAA1 = myfeaturePseudoAAComp(data, d = 15, w = 0.05)
\# Introducing the categorical data into the data frame \#
Subcellular<-c("cytoplasm")
PAA1<-data.frame(Subcellular,PAA1)
PAA1=PAA1[complete.cases(PAA1),]
nums = sample(1:nrow(PAA1), floor(.7*nrow(PAA1)))

```

```

\# (need to do this so models can know that each location is a "possibility"):
training_cytop = PAA1[nums,]
while (0 %in% summary(training_cytop$Subcellular) == TRUE){
  nums = sample(1:nrow(PAA1), floor(.7*nrow(PAA1)))
  training_cytop = PAA1[nums,]
}

\# Create 30% testing set:
$testing_cytop = PAA1[-nums,]$

data = readFASTA("C:/Users/btannan/Desktop/Proteinclean/ER.fas")\#,seqtype= "AA", as.str
PAA2 = myfeaturePseudoAAComp(data, d = 15, w = 0.05)
\# Introducing the categorical data into the data frame \#
Subcellular<-c("ER")
PAA2<-data.frame(Subcellular,PAA2)
PAA2=PAA2[complete.cases(PAA2),]
nums = sample(1:nrow(PAA2), floor(.7*nrow(PAA2)))
### (need to do this so models can know that each location is a "possibility"):
training_ER = PAA2[nums,]
while (0 %in% summary(training_ER$Subcellular) == TRUE){
  nums = sample(1:nrow(PAA2), floor(.7*nrow(PAA2)))
  training_ER = PAA2[nums,]
}

### Create 30% testing set:
testing_ER = PAA2[-nums,]

```

```

data = readFASTA("C:/Users/btannan/Desktop/Proteinclean/golgi.fas")#,seqtype= "AA", as.s
PAA3 = myfeaturePseudoAAComp(data, d = 15, w = 0.05)
#### Introducing the categorical data into the data frame ####
Subcellular<-c("golgi")
PAA3<-data.frame(Subcellular,PAA3)
PAA3=PAA3[complete.cases(PAA3),]
nums = sample(1:nrow(PAA3), floor(.7*nrow(PAA3)))
### (need to do this so models can know that each location is a "possibility"):
training_golgi = PAA3[nums,]
while (0 %in% summary(training_golgi$Subcellular) == TRUE){
  nums = sample(1:nrow(PAA3), floor(.7*nrow(PAA3)))
  training_golgi = PAA3[nums,]
}
### Create 30% testing set:
testing_golgi = PAA3[-nums,]

```

```

data = readFASTA("C:/Users/btannan/Desktop/Proteinclean/lysosome.fas")#,seqtype= "AA", a
PAA4 = myfeaturePseudoAAComp(data, d = 15, w = 0.05)
#### Introducing the categorical data into the data frame ####
Subcellular<-c("lysosome")
PAA4<-data.frame(Subcellular,PAA4)
PAA4=PAA4[complete.cases(PAA4),]
nums = sample(1:nrow(PAA4), floor(.7*nrow(PAA4)))

```

```

### (need to do this so models can know that each location is a "possibility"):
training_lysome = PAA4[nums,]
while (0 %in% summary(training_lysome$Subcellular) == TRUE){
  nums = sample(1:nrow(PAA4), floor(.7*nrow(PAA4)))
  training_lysome = PAA4[nums,]
}

### Create 30% testing set:
testing_lysome = PAA4[-nums,]


data = readFASTA("C:/Users/btannan/Desktop/Proteinclean/mitochondrial.fas")#, seqtype= "A"
PAA5 = myfeaturePseudoAAComp(data, d = 15, w = 0.05)
#### Introducing the categorical data into the data frame ####
Subcellular<-c("mitochondrial")
PAA5<-data.frame(Subcellular,PAA5)
PAA5=PAA5[complete.cases(PAA5),]
nums = sample(1:nrow(PAA5), floor(.7*nrow(PAA5)))
### (need to do this so models can know that each location is a "possibility"):
training_mito = PAA5[nums,]
while (0 %in% summary(training_mito$Subcellular) == TRUE){
  nums = sample(1:nrow(PAA5), floor(.7*nrow(PAA5)))
  training_mito = PAA5[nums,]
}

### Create 30% testing set:
testing_mito = PAA5[-nums,]

```

```

data = readFASTA("C:/Users/btannan/Desktop/Proteinclean/nucleus.fas")#,seqtype= "AA", as
PAA6 = myfeaturePseudoAAComp(data, d = 15, w = 0.05)
#### Introducing the categorical data into the data frame ####
Subcellular<-c("nucleus")
PAA6<-data.frame(Subcellular,PAA6)
PAA6=PAA6[complete.cases(PAA6),]
nums = sample(1:nrow(PAA6), floor(.7*nrow(PAA6)))
### (need to do this so models can know that each location is a "possibility"):
training_nuc = PAA6[nums,]
while (0 %in% summary(training_nuc$Subcellular) == TRUE){
  nums = sample(1:nrow(PAA6), floor(.7*nrow(PAA6)))
  training_nuc = PAA6[nums,]
}
### Create 30% testing set:
testing_nuc = PAA6[-nums,]

```

```

data = readFASTA("C:/Users/btannan/Desktop/Proteinclean/peroxisome.fas")#,seqtype= "AA",
PAA7 = myfeaturePseudoAAComp(data, d = 15, w = 0.05)
#### Introducing the categorical data into the data frame ####
Subcellular<-c("peroxisome")

```

```

PAA7<-data.frame(Subcellular,PAA7)
PAA7=PAA7[complete.cases(PAA7),]
nums = sample(1:nrow(PAA7), floor(.7*nrow(PAA7)))
### (need to do this so models can know that each location is a "possibility"):
training_perox = PAA7[nums,]
while (0 %in% summary(training_perox$Subcellular) == TRUE){
  nums = sample(1:nrow(PAA7), floor(.7*nrow(PAA7)))
  training_perox = PAA7[nums,]
}
### Create 30% testing set:
testing_perox = PAA7[-nums,]

data = readFASTA("C:/Users/btannan/Desktop/Proteinclean/plasma.fas")#, seqtype= "AA", as.
PAA8 = featurePseudoAAComp(data, d = 15, w = 0.05)
#### Introducing the categorical data into the data frame ####
Subcellular<-c("plasma")
PAA8<-data.frame(Subcellular,PAA8)
PAA8=PAA8[complete.cases(PAA8),]
nums = sample(1:nrow(PAA8), floor(.7*nrow(PAA8)))
### (need to do this so models can know that each location is a "possibility"):
training_plas = PAA8[nums,]
while (0 %in% summary(training_plas$Subcellular) == TRUE){
  nums = sample(1:nrow(PAA8), floor(.7*nrow(PAA8)))
  training_plas = PAA8[nums,]
}

```

```

}

### Create 30% testing set:
testing_plas = PAA8[-nums,]


data = readFASTA("C:/Users/btannan/Desktop/Proteinclean/secreted.fas")#, seqtype= "AA", a
PAA9 = myfeaturePseudoAAComp(data, d = 15, w = 0.05)
#### Introducing the categorical data into the data frame ####
Subcellular<-c("secreted")
PAA9<-data.frame(Subcellular,PAA9)
PAA9=PAA9[complete.cases(PAA9),]
nums = sample(1:nrow(PAA9), floor(.7*nrow(PAA9)))
### (need to do this so models can know that each location is a "possibility"):
training_secree = PAA9[nums,]
while (0 %in% summary(training_secree$Subcellular) == TRUE){
  nums = sample(1:nrow(PAA9), floor(.7*nrow(PAA9)))
  training_secree = PAA9[nums,]
}

### Create 30% testing set:
testing_secree = PAA9[-nums,]

```



```

data = readFASTA("C:/Users/btannan/Desktop/Proteinclean/vacuole.fas")#,seqtype= "AA", as
PAA10 = featurePseudoAAComp(data, d = 15, w = 0.05)
#### Introducing the categorical data into the data frame ####
Subcellular<-c("vacuole")
PAA10<-data.frame(Subcellular,PAA10)
PAA10=PAA10[complete.cases(PAA10),]
nums = sample(1:nrow(PAA10), floor(.7*nrow(PAA10)))
### (need to do this so models can know that each location is a "possibility"):
training_vac = PAA10[nums,]
while (0 %in% summary(training_vac$Subcellular) == TRUE){
  nums = sample(1:nrow(PAA10), floor(.7*nrow(PAA10)))
  training_vac = PAA10[nums,]
}
### Create 30% testing set:
testing_vac = PAA10[-nums,]

#### Combining all data and saving them in one file #####

data_training<-rbind(training_cytop,training_ER,training_golgi,training_lysosome,
                     training_mito,training_nuc,training_perox,training_plas,training_se

data_testing<-rbind(testing_cytop,testing_ER,testing_golgi,testing_lysosome,

```

```

testing_mito,testing_nuc,testing_perox,testing_plas,testing_secre,te

save(data_training,file="C:/Users/btannan/Desktop/Proteinclean/train_dataset.Rdata")
save(data_testing,file="C:/Users/btannan/Desktop/Proteinclean/test_dataset.Rdata")


#####
# ADABOOST
x = data_training[,2:36]
y=data_training[,1]
ctrl<-trainControl(method="boot", sampling = "down")
grid_ab<-expand.grid(maxdepth=9, mfinal=150)
set.seed(123)
m_ab1 <- train(x, y, method = "AdaBag",trControl = ctrl,
              metric = "Kappa",
              tuneGrid = grid_ab)
predicted_ab1 = predict(m_ab1, newdata = data_testing)
actual=as.factor(data_testing[,1])
confusion_ab1 = confusionMatrix(data = predicted_ab1, reference = actual)
confusion_ab1

write.csv(as.data.frame.matrix(confusion_ab1$table), file = "C:/Users/btannan/Desktop/Pr
write.csv(as.data.frame.matrix(confusion_ab1$byClass), file = "C:/Users/btannan/Desktop/
write.csv(as.data.frame(confusion_ab1$overall), file = "C:/Users/btannan/Desktop/Protein
### Set c as a confusion matrix, previously calculated and stored

```

```

### (this confusion matrix has rows as predictions and columns as actuals):
c = as.matrix(as.data.frame.matrix(confusion_ab1$table))

### Initialize length 10 MCC vector:
MCC = vector(mode = "numeric", length = 10)

### For each location:
for(k in 1:10){
  A = as.double(c[k,k]) ### A = #(true positives)
  B = as.double(sum(c[k,]) - c[k,k]) ### B = #(false positives)
  C = as.double(sum(c[,k]) - c[k,k]) ### C = #(false negatives)
  D = as.double(sum(c) - A - B - C) ### D = #(true negatives)
  if((((A + B)*(A + C)*
      (D + B)*
      (D + C))^0.5) == 0){ ### if denominator is zero, arbitrarily set as one - MCC will
    MCC[k] = (A*D) - (B*
      C)
  }
  else{MCC[k] = ((A*D) - (B*
      C))/(((A + B)*(A + C)*
      (D + B)*(D + C))^0.5)} ### MCC value by
    formula
    rm(A,B,C,D)
}

### Rounding, setting as column vector:
MCC = round(matrix(MCC, nrow = 10, ncol = 1), digits = 2)

### Set appropriate column name:
colnames(MCC) = "MCC"

# Save results to machine (to be appended to pre-existing statistics by class table):

```

```

write.table(MCC, file = "C:/Users/btannan/Desktop/Proteinclean/DOWNADAPTIVEBOOST_MCCs.csv")
rm(c, MCC)

#####
##### ADAPTIVE BOOSTING WITH UP-SAMPLING #####
x = data_training[,2:36]
y=data_training[,1]
ctrl1<-trainControl(method="boot", sampling = "up")
grid_ab<-expand.grid(maxdepth=9, mfinal=150)
set.seed(123)
m_ab2 <- train(x, y, method = "AdaBag",trControl = ctrl1,
              metric = "Kappa",
              tuneGrid = grid_ab)
predicted_ab2 = predict(m_ab2, newdata = data_testing)
actual=as.factor(data_testing[,1])
confusion_ab2 = confusionMatrix(data = predicted_ab2, reference = actual)
confusion_ab2

write.csv(as.data.frame.matrix(confusion_ab2$table), file = "C:/Users/btannan/Desktop/Protein")
write.csv(as.data.frame.matrix(confusion_ab2$byClass), file = "C:/Users/btannan/Desktop/Protein")
write.csv(as.data.frame(confusion_ab2$overall), file = "C:/Users/btannan/Desktop/Protein")
### Set c as a confusion matrix, previously calculated and stored
### (this confusion matrix has rows as predictions and columns as actuals):
c = as.matrix(as.data.frame.matrix(confusion_ab2$table))
### Initialize length 10 MCC vector:

```

```

MCC = vector(mode = "numeric", length = 10)

### For each location:
for(k in 1:10){
  A = as.double(c[k,k]) ### A = #(true positives)
  B = as.double(sum(c[k,]) - c[k,k]) ### B = #(false positives)
  C = as.double(sum(c[,k]) - c[k,k]) ### C = #(false negatives)
  D = as.double(sum(c) - A - B - C) ### D = #(true negatives)
  if((((A + B)*(A + C)*
      (D + B)*
      (D + C))^0.5) == 0){ ### if denominator is zero, arbitrarily set as one - MCC will
    MCC[k] = (A*D) - (B*
      C)
  }
  else{MCC[k] = ((A*D) - (B*
      C))/(((A + B)*(A + C)*
      (D + B)*(D + C))^0.5)} ### MCC value by
    formula
    rm(A,B,C,D)
}

### Rounding, setting as column vector:
MCC = round(matrix(MCC, nrow = 10, ncol = 1), digits = 2)

### Set appropriate column name:
colnames(MCC) = "MCC"

# Save results to machine (to be appended to pre-existing statistics by class table):
write.table(MCC, file = "C:/Users/btannan/Desktop/Proteinclean/UPADAPTIVEBOOST_MCCs.csv")
rm(c, MCC)

```

```
#####
#####ADAPTIVEBOOSTING#####
x = data_training[,2:36]
y=data_training[,1]
grid_ab<-expand.grid(maxdepth=9, mfinal=150)
set.seed(123)
m_ab3 <- train(x, y, method = "AdaBag",
              metric = "Kappa",
              tuneGrid = grid_ab)
predicted_ab3 = predict(m_ab3, newdata = data_testing)
actual=as.factor(data_testing[,1])
confusion_ab3 = confusionMatrix(data = predicted_ab3, reference = actual)
confusion_ab3

write.csv(as.data.frame.matrix(confusion_ab3$table), file = "C:/Users/btannan/Desktop/Pr
write.csv(as.data.frame.matrix(confusion_ab3$byClass), file = "C:/Users/btannan/Desktop/
write.csv(as.data.frame(confusion_ab3$overall), file = "C:/Users/btannan/Desktop/Protein

### Set c as a confusion matrix, previously calculated and stored
### (this confusion matrix has rows as predictions and columns as actuals):
c = as.matrix(as.data.frame.matrix(confusion_ab3$table))
### Initialize length 10 MCC vector:
MCC = vector(mode = "numeric", length = 10)
### For each location:
for(k in 1:10){
  A = as.double(c[k,k]) ### A = #(true positives)
  B = as.double(sum(c[k,]) - c[k,k]) ### B = #(false positives)
```

```

C = as.double(sum(c[,k]) - c[k,k]) ### C = #(false negatives)
D = as.double(sum(c) - A - B - C) ### D = #(true negatives)
if((((A + B)*(A + C)*
      (D + B)*
      (D + C))^0.5) == 0){ ### if denominator is zero, arbitrarily set as one - MCC will
MCC[k] = (A*D) - (B*
              C)
}
else{MCC[k] = ((A*D) - (B*
              C))/(((A + B)*(A + C)*
              (D + B)*(D + C))^0.5)} ### MCC value by
formula
rm(A,B,C,D)
}

### Rounding, setting as column vector:
MCC = round(matrix(MCC, nrow = 10, ncol = 1), digits = 2)
### Set appropriate column name:
colnames(MCC) = "MCC"
# Save results to machine (to be appended to pre-existing statistics by class table):
write.table(MCC, file = "C:/Users/btannan/Desktop/Proteinclean/ADAPTIVEBOOST_MCCs.csv",
rm(c, MCC)

#####
#####Deep Learning#####
library(h2o)

c1=h2o.init(max_mem_size = "4G",nthreads = -1,ip = "localhost", port = 54321)

```

```
protein_d1 <- h2o.deeplearning(2:36,1,
                                as.h2o(data_training),
                                hidden=c(1024,1024),epochs = 10000)
```

```
pred1 <- h2o.performance(protein_d1, newdata = as.h2o(data_testing))
```

```
?h2o.confusionMatrix
```

```
h2o.confusionMatrix(protein_d1,as.h2o(data_testing))
```

```
h2o.performance(protein_d1)
```

```
h2o.performance(protein_d1, xval = TRUE)
```

```
h2o.confusionMatrix(pred1)
```

```
#####
```

```
library(xgboost)
```

```
x = data_training[,2:36]
```



```

y=data_training[,1]
actual = data_testing[,1]
grid = expand.grid(
  nrounds = 100,
  colsample_bytree = 1,
  min_child_weight = 1,
  eta = c(0.1, 0.3, 0.5), #0.3 is default,
  gamma = c(0.25, 0.5),
  subsample = 1,
  max_depth = c(3))

```

```
#####
```

```
##### XGBoost with Down Sampling#####
```

```
#####
```

```
x = data_training[,2:36]
```

```
y=data_training[,1]
```

```
actual = data_testing[,1]
```

```

grid = expand.grid(
  nrounds = 100,
  colsample_bytree = 1,
  min_child_weight = 1,
  eta = c(0.1, 0.3, 0.5), #0.3 is default,
  gamma = c(0.25, 0.5),
  subsample = 1,
  max_depth = c(3))

```

```

cntrl = trainControl(method = "cv", number = 5,
                     verboseIter = TRUE, returnData = FALSE,
                     returnResamp = "final", sampling = "down")

set.seed(123)

downtrain.xgb = train(x , y ,
                     trControl = cntrl,
                     tuneGrid = grid,
                     method = "xgbTree",
                     metric = "Kappa"
)

downpredicted_XGBoost = predict(downtrain.xgb, newdata = data_testing)
confusion_downXGboost = confusionMatrix(data = downpredicted_XGBoost, reference = actual)
confusion_downXGboost

write.csv(as.data.frame.matrix(confusion_downXGboost$table), file = "C:/Users/btannan/Desktop/
write.csv(as.data.frame.matrix(confusion_downXGboost$byClass), file = "C:/Users/btannan/Desktop/
write.csv(as.data.frame(confusion_downXGboost$overall), file = "C:/Users/btannan/Desktop/

### Set c as a confusion matrix, previously calculated and stored
### (this confusion matrix has rows as predictions and columns as actuals):
c = as.matrix(as.data.frame.matrix(confusion_downXGboost$table))

### Initialize length 10 MCC vector:

MCC = vector(mode = "numeric", length = 10)

### For each location:
for(k in 1:10){
  A = as.double(c[k,k]) ### A = #(true positives)
  B = as.double(sum(c[k,]) - c[k,k]) ### B = #(false positives)

```

```

C = as.double(sum(c[,k]) - c[k,k]) ### C = #(false negatives)
D = as.double(sum(c) - A - B - C) ### D = #(true negatives)
if((((A + B)*(A + C)*
      (D + B)*
      (D + C))^0.5) == 0){ ### if denominator is zero, arbitrarily set as one - MCC will
MCC[k] = (A*D) - (B*
              C)
}
else{MCC[k] = ((A*D) - (B*
              C))/((((A + B)*(A + C)*
              (D + B)*(D + C))^0.5)} ### MCC value by
formula
rm(A,B,C,D)
}

### Rounding, setting as column vector:
MCC = round(matrix(MCC, nrow = 10, ncol = 1), digits = 2)

### Set appropriate column name:
colnames(MCC) = "MCC"

# Save results to machine (to be appended to pre-existing statistics by class table):
write.table(MCC, file = "C:/Users/btannan/Desktop/Proteinclean/downxgboost_MCCs.csv", se
rm(c, MCC)

#####
##### XGBoost with Up Sampling#####
#####
x = data_training[,2:36]

```

```

y=data_training[,1]
actual = data_testing[,1]
grid = expand.grid(
  nrounds = 100,
  colsample_bytree = 1,
  min_child_weight = 1,
  eta = c(0.1, 0.3, 0.5), #0.3 is default,
  gamma = c(0.25, 0.5),
  subsample = 1,
  max_depth = c(3))

cntrl = trainControl(method = "cv",number = 5,
                     verboseIter = TRUE,returnData = FALSE,
                     returnResamp = "final", sampling = "up")

set.seed(123)

uptrain.xgb = train(x , y ,trControl = cntrl,
                   tuneGrid = grid,
                   method = "xgbTree",
                   metric = "Kappa"
)

uppredicted_XGBoost = predict(uptrain.xgb, newdata = data_testing)
actual=as.factor(data_testing[,1])
confusion_upXGboost = confusionMatrix(data = uppredicted_XGBoost, reference = actual)
confusion_upXGboost

write.csv(as.data.frame.matrix(confusion_upXGboost$table), file = "C:/Users/btannan/Desktop/ConfusionMatrix.csv")
write.csv(as.data.frame.matrix(confusion_upXGboost$byClass), file = "C:/Users/btannan/Desktop/ConfusionMatrixByClass.csv")

```

```

write.csv(as.data.frame(confusion_upXGboost$overall), file = "C:/Users/btannan/Desktop/P
### Set c as a confusion matrix, previously calculated and stored
### (this confusion matrix has rows as predictions and columns as actuals):
c = as.matrix(as.data.frame.matrix(confusion_upXGboost$table))
### Initialize length 10 MCC vector:
MCC = vector(mode = "numeric", length = 10)
### For each location:
for(k in 1:10){
  A = as.double(c[k,k]) ### A = #(true positives)
  B = as.double(sum(c[k,]) - c[k,k]) ### B = #(false positives)
  C = as.double(sum(c[,k]) - c[k,k]) ### C = #(false negatives)
  D = as.double(sum(c) - A - B - C) ### D = #(true negatives)
  if((((A + B)*(A + C)*
      (D + B)*
      (D + C))^0.5) == 0){ ### if denominator is zero, arbitrarily set as one - MCC will
MCC[k] = (A*D) - (B*
      C)
  }
  else{MCC[k] = ((A*D) - (B*
      C))/(((A + B)*(A + C)*
      (D + B)*(D + C))^0.5)} ### MCC value by
formula
rm(A,B,C,D)
}
### Rounding, setting as column vector:
MCC = round(matrix(MCC, nrow = 10, ncol = 1), digits = 2)
### Set appropriate column name:

```

```

colnames(MCC) = "MCC"

# Save results to machine (to be appended to pre-existing statistics by class table):
write.table(MCC, file = "C:/Users/btannan/Desktop/Proteinclean/upxgboost_MCCs.csv", sep
rm(c, MCC)

##### XGBoost #####
x = data_training[,2:36]
y=data_training[,1]
actual = data_testing[,1]
grid = expand.grid(
  nrounds = 100,
  colsample_bytree = 1,
  min_child_weight = 1,
  eta = c(0.1, 0.3, 0.5), #0.3 is default,
  gamma = c(0.25, 0.5),
  subsample = 1,
  max_depth = c(3))

cntrl = trainControl(method = "cv",number = 5,
                     verboseIter = TRUE,returnData = FALSE,
                     returnResamp = "final")

set.seed(123)

train.xgb = train(x , y ,
                  trControl = cntrl,
                  tuneGrid = grid,
                  method = "xgbTree",
                  metric = "Kappa"

```

```

)

predicted_XGBoost = predict(train.xgb, newdata = data_testing)
actual=as.factor(data_testing[,1])
confusion_XGboost = confusionMatrix(data = predicted_XGBoost, reference = actual)
confusion_XGboost

write.csv(as.data.frame.matrix(confusion_XGboost$table), file = "C:/Users/btannan/Desktop/Pro
write.csv(as.data.frame.matrix(confusion_XGboost$byClass), file = "C:/Users/btannan/Desktop/Pro
write.csv(as.data.frame(confusion_XGboost$overall), file = "C:/Users/btannan/Desktop/Pro
### Set c as a confusion matrix, previously calculated and stored
### (this confusion matrix has rows as predictions and columns as actuals):
c = as.matrix(as.data.frame.matrix(confusion_XGboost$table))
### Initialize length 10 MCC vector:
MCC = vector(mode = "numeric", length = 10)

### For each location:
for(k in 1:10){
  A = as.double(c[k,k]) ### A = #(true positives)
  B = as.double(sum(c[k,]) - c[k,k]) ### B = #(false positives)
  C = as.double(sum(c[,k]) - c[k,k]) ### C = #(false negatives)
  D = as.double(sum(c) - A - B - C) ### D = #(true negatives)
  if((((A + B)*(A + C)*
    (D + B)*
    (D + C))^0.5) == 0){ ### if denominator is zero, arbitrarily set as one - MCC will
    MCC[k] = (A*D) - (B*
      C)
  }
}

```

```

else{MCC[k] = ((A*D) - (B*
                                C))/(((A + B)*(A + C)*
                                (D + B)*(D + C))^.5)} ### MCC value by

formula

rm(A,B,C,D)
}

### Rounding, setting as column vector:
MCC = round(matrix(MCC, nrow = 10, ncol = 1), digits = 2)
### Set appropriate column name:
colnames(MCC) = "MCC"

# Save results to machine (to be appended to pre-existing statistics by class table):
write.table(MCC, file = "C:/Users/btannan/Desktop/Proteinclean/xgboost_MCCs.csv", sep =
rm(c, MCC)

#####
##### RANDOM FOREST WITH DOWN SAMPLING#####
x = data_training[,2:36]
y=data_training[,1]

ctrl<-trainControl(method="oob", sampling = "down")
grid_rf<-expand.grid(.mtry=c(2, 4, 8))

set.seed(123)

m_rf2 <- train(x, y, method = "rf", metric = "Kappa", trControl = ctrl, trees=50)
m_rf2

```



```

predicted_RFOREST2 = predict(m_rf2, newdata = data_testing)

confusion_RFOREST2 = confusionMatrix(data = predicted_RFOREST2, reference = actual)
confusion_RFOREST2

actual=as.factor(data_testing[,1])

write.csv(as.data.frame.matrix(confusion_RFOREST2$table), file = "C:/Users/btannan/Desktop/Pr
write.csv(as.data.frame.matrix(confusion_RFOREST2$byClass), file = "C:/Users/btannan/Des
write.csv(as.data.frame(confusion_RFOREST2$overall), file = "C:/Users/btannan/Desktop/Pr

### Set c as a confusion matrix, previously calculated and stored ### (this confusion ma

c = as.matrix(as.data.frame.matrix(confusion_RFOREST2$table))
### Initialize length 10 MCC vector:

MCC = vector(mode = "numeric", length = 10)

### For each location:

for(k in 1:10){
A = as.double(c[k,k]) ### A = #(true positives)
B = as.double(sum(c[k,]) - c[k,k]) ### B = #(false positives)
C = as.double(sum(c[,k]) - c[k,k]) ### C = #(false negatives)
D = as.double(sum(c) - A - B - C) ### D = #(true negatives)
if((((A + B)*(A + C)* (D + B)*(D + C))^0.5) == 0){

```

```

### if denominator is zero, arbitrarily set as one - MCC will come out to be zero MCC[k]
}

else{MCC[k] = ((A*D) - (B*C))/(((A + B)*(A + C)*(D + B)*(D + C))^.5)}

### MCC value by formula rm(A,B,C,D)
}

### Rounding, setting as column vector:

MCC = round(matrix(MCC, nrow = 10, ncol = 1), digits = 2)

### Set appropriate column name:

colnames(MCC) = "MCC"

# Save results to machine (to be appended to pre-existing statistics by class table):

write.table(MCC, file = "C:/Users/btannan/Desktop/Proteinclean/DOWNRANDOMFOREST2_MCCs.csv",
as.csv = TRUE,
col.names = FALSE,
row.names = FALSE)

rm(c, MCC)

#####
#####RANDOM FOREST WITH DOWN SAMPLING#####

x = data_training[,2:36]
y=data_training[,1]

ctrl<-trainControl(method="oob")
grid_rf<-expand.grid(.mtry=c( 4, 8))

```

```

set.seed(123)

m_rf1 <- train(x, y, method = "rf", metric = "Kappa", trControl = ctrl, tuneGrid = grid_
m_rf1

predicted_RFOREST1 = predict(m_rf1, newdata = data_testing)
actual=as.factor(data_testing[,1])
confusion_RFOREST1 = confusionMatrix(data = predicted_RFOREST1, reference = actual)
confusion_RFOREST1

write.csv(as.data.frame.matrix(confusion_RFOREST1$table), file = "C:/Users/btannan/Deskt

write.csv(as.data.frame.matrix(confusion_RFOREST1$byClass), file = "C:/Users/btannan/Des
write.csv(as.data.frame(confusion_RFOREST1$overall), file = "C:/Users/btannan/Desktop/Pr

### Set c as a confusion matrix, previously calculated and stored ### (this confusion ma

c = as.matrix(as.data.frame.matrix(confusion_RFOREST1$table)) ### Initialize length 10 M

MCC = vector(mode = "numeric", length = 10)

### For each location:

```

```

for(k in 1:10){
  A = as.double(c[k,k]) ### A = #(true positives)
  B = as.double(sum(c[k,]) - c[k,k]) ### B = #(false positives)
  C = as.double(sum(c[,k]) - c[k,k]) ### C = #(false negatives)
  D = as.double(sum(c) - A - B - C) ### D = #(true negatives)
  if((((A + B)*(A + C)* (D + B)*(D + C))^0.5) == 0){
    ### if denominator is zero, arbitrarily set as one - MCC will come out to be zero MCC
  }

  else{MCC[k] = ((A*D) - (B*C))/((((A + B)*(A + C)*(D + B)*(D + C))^0.5)}

  ### MCC value by formula rm(A,B,C,D)
}

### Rounding, setting as column vector:

MCC = round(matrix(MCC, nrow = 10, ncol = 1), digits = 2)

### Set appropriate column name:

colnames(MCC) = "MCC"

# Save results to machine (to be appended to pre-existing statistics by class table):

write.table(MCC, file = "C:/Users/btannan/Desktop/Proteinclean/RANDOMFOREST_MCCs.csv", s
rm(c, MCC)

```

```
#####
#####
x = data_training[,2:36]
y=data_training[,1]

ctrl<-trainControl(method="oob", sampling = "up")
grid_rf<-expand.grid(.mtry=c(2, 4, 8))

set.seed(123)
m_rf4 <- train(x, y, method = "rf", metric = "Kappa", trControl = ctrl, trees=50)
m_rf4

predicted_RFOREST4 = predict(m_rf4, newdata = data_testing)
actual=as.factor(data_testing[,1])
confusion_RFOREST4 = confusionMatrix(data = predicted_RFOREST4, reference = actual)
confusion_RFOREST4

write.csv(as.data.frame.matrix(confusion_RFOREST4$table), file = "C:/Users/btannan/Desktop/Pr
write.csv(as.data.frame.matrix(confusion_RFOREST4$byClass), file = "C:/Users/btannan/Desktop/Pr
write.csv(as.data.frame(confusion_RFOREST4$overall), file = "C:/Users/btannan/Desktop/Pr

### Set c as a confusion matrix, previously calculated and stored ### (this confusion ma
```

```

c = as.matrix(as.data.frame.matrix(confusion_RFOREST4$table))
### Initialize length 10 MCC vector:

MCC = vector(mode = "numeric", length = 10)

### For each location:

for(k in 1:10){
  A = as.double(c[k,k]) ### A = #(true positives)
  B = as.double(sum(c[k,]) - c[k,k]) ### B = #(false positives)
  C = as.double(sum(c[,k]) - c[k,k]) ### C = #(false negatives)
  D = as.double(sum(c) - A - B - C) ### D = #(true negatives)
  if((((A + B)*(A + C)* (D + B)*(D + C))^0.5) == 0){
    ### if denominator is zero, arbitrarily set as one - MCC will come out to be zero MCC
  }

  else{MCC[k] = ((A*D) - (B*C))/((((A + B)*(A + C)*(D + B)*(D + C))^0.5)}

  ### MCC value by formula rm(A,B,C,D)
}

### Rounding, setting as column vector:

MCC = round(matrix(MCC, nrow = 10, ncol = 1), digits = 2)

### Set appropriate column name:

```

```
colnames(MCC) = "MCC"
```

```
# Save results to machine (to be appended to pre-existing statistics by class table):
```

```
write.table(MCC, file = "C:/Users/btannan/Desktop/Proteinclean/UPRANDOMFOREST4_MCCs.csv"
```

```
rm(c, MCC)
```

```
#####
```

```
##### BAGGED TREES#####
```

```
x = data_training[,2:36]
```

```
y=data_training[,1]
```

```
ctrl <- trainControl(method = "boot",classProbs = TRUE)
```

```
grid_rf<-expand.grid(.mtry=c(2, 4, 8))
```

```
set.seed(5627)
```

```
original_train1 <- train(x, y, method = "treebag", metric = "Kappa", trControl = ctrl,
```

```
original_train1
```

```

caretpred_bagging1 = predict(original_train1, newdata = data_testing)
actual=as.factor(data_testing[,1])
bagged_confusionmatrix = confusionMatrix(data = caretpred_bagging1, reference = actual)
bagged_confusionmatrix

write.csv(as.data.frame.matrix(bagged_confusionmatrix$table), file = "C:/Users/btannan/D
write.csv(as.data.frame.matrix(bagged_confusionmatrix$byClass), file = "C:/Users/btannan
write.csv(as.data.frame(bagged_confusionmatrix$overall), file = "C:/Users/btannan/Desкто

### Set c as a confusion matrix, previously calculated and stored ### (this confusion ma
c = as.matrix(as.data.frame.matrix(bagged_confusionmatrix$table)) ### Initialize length
MCC = vector(mode = "numeric", length = 10)

### For each location:
for(k in 1:10){
  A = as.double(c[k,k]) ### A = #(true positives)
  B = as.double(sum(c[k,]) - c[k,k]) ### B = #(false positives)
  C = as.double(sum(c[,k]) - c[k,k]) ### C = #(false negatives)
  D = as.double(sum(c) - A - B - C) ### D = #(true negatives)
  if((((A + B)*(A + C)* (D + B)*(D + C))^0.5) == 0){
    ### if denominator is zero, arbitrarily set as one - MCC will come out to be zero MCC
  }

  else{MCC[k] = ((A*D) - (B*C))/((((A + B)*(A + C)*(D + B)*(D + C))^0.5)}
}

```



```

    ### MCC value by formula rm(A,B,C,D)
}

### Rounding, setting as column vector:
MCC = round(matrix(MCC, nrow = 10, ncol = 1), digits = 2)
### Set appropriate column name:
colnames(MCC) = "MCC"
# Save results to machine (to be appended to pre-existing statistics by class table):

write.table(MCC, file = "C:/Users/btannan/Desktop/Proteinclean/BAGGING_MCCs.csv", sep =
rm(c, MCC)

#####
#####BAGGED DOWN SAMPLING#####
x = data_training[,2:36]
y=data_training[,1]

ctrl <- trainControl(method = "boot", classProbs = TRUE, sampling = "down")
grid_rf<-expand.grid(.mtry=c(2, 4, 8))

set.seed(5627)
down_inside <- train(x, y, method = "treebag", metric = "Kappa", trControl = ctrl, tree
down_inside

```

```

caretpred_bagging2 = predict(down_inside, newdata = data_testing)
actual=as.factor(data_testing[,1])
downbagged_confusionmatrix = confusionMatrix(data = caretpred_bagging2, reference = actual)
downbagged_confusionmatrix

write.csv(as.data.frame.matrix(downbagged_confusionmatrix$table), file = "C:/Users/btannan/Desktop/Downbagged_ConfusionMatrix.csv")
write.csv(as.data.frame.matrix(downbagged_confusionmatrix$byClass), file = "C:/Users/btannan/Desktop/Downbagged_ConfusionMatrix_byClass.csv")
write.csv(as.data.frame(downbagged_confusionmatrix$overall), file = "C:/Users/btannan/Desktop/Downbagged_ConfusionMatrix_overall.csv")

### Set c as a confusion matrix, previously calculated and stored ### (this confusion matrix is for the downboot data)

c = as.matrix(as.data.frame.matrix(downbootconfusionMatrix$table)) ### Initialize length of MCC vector
MCC = vector(mode = "numeric", length = 10)

### For each location:
for(k in 1:10){
  A = as.double(c[k,k]) ### A = #(true positives)
  B = as.double(sum(c[k,]) - c[k,k]) ### B = #(false positives)
  C = as.double(sum(c[,k]) - c[k,k]) ### C = #(false negatives)
  D = as.double(sum(c) - A - B - C) ### D = #(true negatives)
  if((((A + B)*(A + C)*(D + B)*(D + C))^0.5) == 0){
    ### if denominator is zero, arbitrarily set as one - MCC will come out to be zero MCC[k] = 0
  }

  else{MCC[k] = ((A*D) - (B*C))/((((A + B)*(A + C)*(D + B)*(D + C))^0.5)}
}

```

```

    ### MCC value by formula rm(A,B,C,D)
}

### Rounding, setting as column vector:
MCC = round(matrix(MCC, nrow = 10, ncol = 1), digits = 2)

### Set appropriate column name:
colnames(MCC) = "MCC"

# Save results to machine (to be appended to pre-existing statistics by class table):

write.table(MCC, file = "C:/Users/btannan/Desktop/Proteinclean/downbootbagging_MCCs.csv")
rm(c, MCC)

#####
#####BAGGED UP SAMPLING#####
x = data_training[,2:36]
y=data_training[,1]

ctrl <- trainControl(method = "boot", classProbs = TRUE, sampling = "up")
grid_rf<-expand.grid(.mtry=c(2, 4, 8))

set.seed(5627)

```

```

up_inside <- train(x, y, method = "treebag", metric = "Kappa", trControl = ctrl, trees=
up_inside

caretpred_bagging3 = predict(up_inside, newdata = data_testing)
actual=as.factor(data_testing[,1])
upbagged_confusionmatrix = confusionMatrix(data = caretpred_bagging3, reference = actual)
upbagged_confusionmatrix

write.csv(as.data.frame.matrix(upbagged_confusionmatrix$table), file = "C:/Users/btannan
write.csv(as.data.frame.matrix(upbagged_confusionmatrix$byClass), file = "C:/Users/btann
write.csv(as.data.frame(upbagged_confusionmatrix$overall), file = "C:/Users/btannan/Desk

### Set c as a confusion matrix, previously calculated and stored
### (this confusion matrix has rows as predictions and columns as actuals):

c = as.matrix(as.data.frame.matrix(upbootconfusionMatrix$table))
### Initialize length 10 MCC vector:
MCC = vector(mode = "numeric", length = 10)

### For each location:
for(k in 1:10){
  A = as.double(c[k,k]) ### A = #(true positives)
  B = as.double(sum(c[k,]) - c[k,k]) ### B = #(false positives)
  C = as.double(sum(c[,k]) - c[k,k]) ### C = #(false negatives)

```

```

D = as.double(sum(c) - A - B - C) ### D = #(true negatives)
if((((A + B)*(A + C)* (D + B)*(D + C))^0.5) == 0){
  ### if denominator is zero, arbitrarily set as one - MCC will come out to be zero MCC
}

else{MCC[k] = ((A*D) - (B*C))/((((A + B)*(A + C)*(D + B)*(D + C))^0.5)}

  ### MCC value by formula rm(A,B,C,D)
}

### Rounding, setting as column vector:
MCC = round(matrix(MCC, nrow = 10, ncol = 1), digits = 2)
### Set appropriate column name:
colnames(MCC) = "MCC"

# Save results to machine (to be appended to pre-existing statistics by class table):
write.table(MCC, file = "C:/Users/btannan/Desktop/Proteinclean/upbootbagging_MCCs.csv",
rm(c, MCC)

#####
#####CROSS-VALIDATED BAGGED #####
x = data_training[,2:36]
y=data_training[,1]

ctrl <- trainControl(method = "repeatedcv",classProbs = TRUE)
grid_rf<-expand.grid(.mtry=c(2, 4, 8))

```

```

set.seed(5627)

orig_train <- train(x, y, method = "treebag", metric = "Kappa", trControl = ctrl, trees = 100)

orig_train

caretpred_cvbagging = predict(orig_train, newdata = data_testing)
actual=as.factor(data_testing[,1])
cvbagged_confusionmatrix = confusionMatrix(data = caretpred_cvbagging, reference = actual)
cvbagged_confusionmatrix

write.csv(as.data.frame.matrix(cvbagged_confusionmatrix$table), file = "C:/Users/btannan/Desktop/orig_train.csv")
write.csv(as.data.frame.matrix(cvbagged_confusionmatrix$byClass), file = "C:/Users/btannan/Desktop/orig_train_byClass.csv")
write.csv(as.data.frame(cvbagged_confusionmatrix$overall), file = "C:/Users/btannan/Desktop/orig_train_overall.csv")
### Set c as a confusion matrix, previously calculated and stored ### (this confusion matrix is for the orig_train data)

c = as.matrix(as.data.frame.matrix(cvbagged_confusionmatrix$table)) ### Initialize length of vector to 10

MCC = vector(mode = "numeric", length = 10)

### For each location:
for(k in 1:10){
  A = as.double(c[k,k]) ### A = #(true positives)
  B = as.double(sum(c[k,]) - c[k,k]) ### B = #(false positives)
  C = as.double(sum(c[,k]) - c[k,k]) ### C = #(false negatives)
  D = as.double(sum(c) - A - B - C) ### D = #(true negatives)
}

```

```

if((((A + B)*(A + C)* (D + B)*(D + C))^0.5) == 0){
    ### if denominator is zero, arbitrarily set as one - MCC will come out to be zero MCC
}

else{MCC[k] = ((A*D) - (B*C))/((((A + B)*(A + C)*(D + B)*(D + C))^0.5)}

    ### MCC value by formula rm(A,B,C,D)
}

### Rounding, setting as column vector:
MCC = round(matrix(MCC, nrow = 10, ncol = 1), digits = 2)
### Set appropriate column name:
colnames(MCC) = "MCC"

# Save results to machine (to be appended to pre-existing statistics by class table):
write.table(MCC, file = "C:/Users/btannan/Desktop/Proteinclean/CVBAGGING_MCCs.csv", sep
rm(c, MCC)

#####
#####CROSS-VALIDATED BAGGED DOWN SAMPLING#####

x = data_training[,2:36]
y=data_training[,1]

ctrl <- trainControl(method = "boot", classProbs = TRUE, sampling = "down")
grid_rf<-expand.grid(.mtry=c(2, 4, 8))

```

```

set.seed(5627)

orig_train1 <- train(x, y, method = "treebag", metric = "Kappa", trControl = ctrl, tree
orig_train1

caretpred_cvbagging1 = predict(orig_train1, newdata = data_testing)
actual=as.factor(data_testing[,1])
downcvbagged_confusionmatrix = confusionMatrix(data = caretpred_cvbagging1, reference =
downcvbagged_confusionmatrix

write.csv(as.data.frame.matrix(downcvbagged_confusionmatrix$table), file = "C:/Users/bta
write.csv(as.data.frame.matrix(downcvbagged_confusionmatrix$byClass), file = "C:/Users/b
write.csv(as.data.frame(downcvbagged_confusionmatrix$overall), file = "C:/Users/btannan/
### Set c as a confusion matrix, previously calculated and stored ### (this confusion ma

c = as.matrix(as.data.frame.matrix(downcvbagged_confusionmatrix$table)) ### Initialize 1

MCC = vector(mode = "numeric", length = 10)

### For each location:
for(k in 1:10){
  A = as.double(c[k,k]) ### A = #(true positives)
  B = as.double(sum(c[k,]) - c[k,k]) ### B = #(false positives)
  C = as.double(sum(c[,k]) - c[k,k]) ### C = #(false negatives)
  D = as.double(sum(c) - A - B - C) ### D = #(true negatives)
  if((((A + B)*(A + C)* (D + B)*(D + C))^0.5) == 0){
    ### if denominator is zero, arbitrarily set as one - MCC will come out to be zero MCC

```



```

}

else{MCC[k] = ((A*D) - (B*C))/(((A + B)*(A + C)*(D + B)*(D + C))^.5)}

### MCC value by formula rm(A,B,C,D)
}

### Rounding, setting as column vector:
MCC = round(matrix(MCC, nrow = 10, ncol = 1), digits = 2)

### Set appropriate column name:
colnames(MCC) = "MCC"

# Save results to machine (to be appended to pre-existing statistics by class table):
write.table(MCC, file = "C:/Users/btannan/Desktop/Proteinclean/DOWNCVBAGGING_MCCs.csv",
rm(c, MCC)

#####
#####CROSS-VALIDATED BAGGED UP SAMPLING#####

x = data_training[,2:36]
y=data_training[,1]

ctrl <- trainControl(method = "boot", classProbs = TRUE, sampling = "up")
grid_rf<-expand.grid(.mtry=c(2, 4, 8))

```

```

set.seed(5627)

orig_train2 <- train(x, y, method = "treebag", metric = "Kappa", trControl = ctrl, tree
orig_train2

caretpred_cvbagging1 = predict(orig_train2, newdata = data_testing)
actual=as.factor(data_testing[,1])
UPcvbagged_confusionmatrix = confusionMatrix(data = caretpred_cvbagging1, reference = ac
UPcvbagged_confusionmatrix

write.csv(as.data.frame.matrix(UPcvbagged_confusionmatrix$table), file = "C:/Users/btann
write.csv(as.data.frame.matrix(UPcvbagged_confusionmatrix$byClass), file = "C:/Users/bta
write.csv(as.data.frame(UPcvbagged_confusionmatrix$overall), file = "C:/Users/btannan/De
### Set c as a confusion matrix, previously calculated and stored ### (this confusion ma

c = as.matrix(as.data.frame.matrix(UPcvbagged_confusionmatrix$table)) ### Initialize len

MCC = vector(mode = "numeric", length = 10)

### For each location:
for(k in 1:10){
  A = as.double(c[k,k]) ### A = #(true positives)
  B = as.double(sum(c[k,]) - c[k,k]) ### B = #(false positives)
  C = as.double(sum(c[,k]) - c[k,k]) ### C = #(false negatives)
  D = as.double(sum(c) - A - B - C) ### D = #(true negatives)
  if((((A + B)*(A + C)* (D + B)*(D + C))^0.5) == 0){
    ### if denominator is zero, arbitrarily set as one - MCC will come out to be zero MC
  }
}

```

```

else{MCC[k] = ((A*D) - (B*C))/(((A + B)*(A + C)*(D + B)*(D + C))^.5)}

### MCC value by formula rm(A,B,C,D)
}

### Rounding, setting as column vector:
MCC = round(matrix(MCC, nrow = 10, ncol = 1), digits = 2)

### Set appropriate column name:
colnames(MCC) = "MCC"

# Save results to machine (to be appended to pre-existing statistics by class table):
write.table(MCC, file = "C:/Users/btannan/Desktop/Proteinclean/UPCVBAGGING_MCCs.csv", se
rm(c, MCC)

#####
#####AutoML#####

library(h2o)
h2o.init()

y <- "Subcellular"
x <- setdiff(names(data_training), y)

train[,y] <- as.factor(data_training[,y])
test[,y] <- as.factor(data_testing[,y])

```

```
nfolds <- 5
```

```
auto=h2o.automl(x = x, y = y,  
               as.h2o(data_training),project_name = "autodeep",  
               include_algos = c("GLM","XGBoost","deeplearning"))
```

```
perf_auto_test <- h2o.predict(auto, newdata = as.h2o(data_testing))
```

References

- Alberts, B., Johnson, A., Lewis, J., Raff, M., Roberts, K., & Walter, P. (2002). *Drosophila and the molecular genetics of pattern formation: Genesis of the body plan*. In *Molecular biology of the cell. 4th edition*. Garland Science.
- Chou, K.-C., & Shen, H.-B. (2006). Hum-ploc: a novel ensemble classifier for predicting human protein subcellular localization. *Biochemical and biophysical research communications*, 347(1), 150–157.
- Chou, K.-C., & Shen, H.-B. (2007). Recent progress in protein subcellular location prediction. *Analytical biochemistry*, 370(1), 1.
- Chou, K.-C., & Shen, H.-B. (2010). A new method for predicting the subcellular localization of eukaryotic proteins with both single and multiple sites: Euk-mploc 2.0. *PloS one*, 5(4).
- Emanuelsson, O. (2000). Nielsen h, brunak s, and von heijne g. *Predicting subcellular localization of proteins based on their N-terminal amino acid sequence*. *J Mol Biol*, 300, 1005–1016.
- Glory, E., & Murphy, R. F. (2007). Automated subcellular location determination and high-throughput microscopy. *Developmental cell*, 12(1), 7–16.
- Meissner, B., Rogalski, T., Viveiros, R., Warner, A., Plastino, L., Lorch, A., ... Moerman, D. G. (2011). Determining the sub-cellular localization of proteins within caenorhabditis elegans body wall muscle. *PloS one*, 6(5).
- Munyon, J. (2015). *Predicting fungal protein subcellular location*. May.
- Neizer-Ashun, K., Yu, F., Meinken, J., Min, X., & Chang, G. (2015). Prediction of plant protein subcellular locations. In *7th international conference on bioinformatics and computational biology (bicob 2015), honolulu, hawaii, usa* (pp. 91–96).
- Webb, C. D., Decatur, A., Teleman, A., & Losick, R. (1995). Use of green fluorescent protein for visualization of cell-specific gene expression and subcellular protein localization during sporulation in bacillus subtilis. *Journal of bacteriology*, 177(20), 5906–5911.